

Informe de pruebas unitarias

GraphRAG Auditor — Backend

Suite: tests/unit — 116 pruebas

Fecha de ejecución: 12 de julio de 2026

1. Alcance de la prueba

Este informe documenta la ejecución de la suite de pruebas unitarias del backend de GraphRAG Auditor, ubicada en tests/unit. La suite verifica la lógica determinista que sostiene el pipeline de auditoría de citas: la que decide qué texto se analiza, a qué fuente pertenece cada cita, qué evidencia se entrega al juez y bajo qué condiciones se emite un veredicto favorable. Esa lógica se comprueba con independencia del modelo de lenguaje y de los servicios externos, que se sustituyen por dobles de prueba.

Los ocho componentes cubiertos son los siguientes:

Componente	Módulo bajo prueba	Responsabilidad verificada	Pruebas
PDFExtractionService	app/services/ingesta/pdf_service.py	Validación del archivo, extracción de texto y detección de secciones	19
EntidadExtractionService	app/services/grafos/extraccion_service.py	Detección de citas APA, delimitación de la bibliografía y troceado del texto	31
GrafoCargaService	app/services/grafos/grafos_carga_service.py	Vinculación automática de cada cita con su referencia	12
RecuperacionService	app/services/recuperacion/recuperacion_service.py	Recuperación híbrida de evidencia (grafo y pgvector)	8
AuditoriaService	app/services/auditoria/auditoria_service.py	Política de emisión del veredicto por cita	14
metricas_service	app/services/evaluacion/metricas_service.py	Kappa de Cohen, F1 y matriz de confusión	12
doi_utils	app/services/externo/doi_utils.py	Normalización de DOI e identidad de la	12

		fuelle	
limpiar_fragmento	app/core/texto.py	Saneado del markdown extraído del PDF	8

Cada prueba de este informe se documenta con su descripción, su código fuente, la entrada con la que se ejerció el componente, la salida que este devolvió y el resultado de la comprobación. Las entradas y salidas registradas son las de la ejecución real: se capturaron instrumentando las llamadas al código bajo prueba durante la corrida, no se transcribieron a mano.

2. Documento de prueba

Las pruebas no emplean texto inventado. Todo lo que se afirma sobre citas, referencias, secciones y DOI se comprueba contra un documento real, incluido en el repositorio junto a las pruebas:

Atributo	Valor medido en la ejecución
Archivo	tests/unit/Informe de tesis_AlcaldeLavadoMatias (1).pdf
Título	Informe de tesis — GraphRAG Auditor
Autor	Matías Alcalde Lavado
Formato	PDF con capa de texto
Páginas	67
Texto extraído	121 193 caracteres de markdown
Cuerpo sin anexos	96 093 caracteres (se descartan 25 100 de anexos)
Sección de referencias	Presente: «7. Referencias bibliográficas», 34 635 caracteres
DOI en la bibliografía	37
Citas APA detectadas	Más de 30 en el cuerpo del documento

El uso de un documento real es deliberado: la detección de citas, la delimitación de la bibliografía y el troceado del texto dependen de la forma en que un PDF académico se convierte a markdown —encabezados, guiones suaves, saltos de línea, apellidos partidos— y ninguna de esas condiciones se reproduce fielmente con texto sintético. Las citas verificadas en las pruebas, como «(Edge et al., 2024)» o «(Klesel y Wittmann, 2025)», y los DOI empleados, como 10.3390/educsci11040173, proceden de la bibliografía real del documento.

El PDF se copia a un directorio temporal antes de cada sesión, ya que la extracción deja un archivo .md de caché junto al original; de este modo el repositorio no se altera.

3. Aislamiento de dependencias

Ninguna prueba de la suite abre una conexión de red, consulta una base de datos ni invoca un modelo de lenguaje. Toda dependencia externa se sustituye por un doble de prueba mediante unittest.mock.patch. La tabla siguiente detalla el tratamiento de cada una.

Dependencia externa	Tratamiento en las pruebas
Modelo de lenguaje (Grok / OpenAI)	Sustituido. En AuditoriaService se parchea _llamar_llm y su respuesta se fija en la prueba. Las funciones de extracción que invocan al LLM no se

	ejercitan: se prueba la lógica determinista que decide qué texto se le envía y cómo se parsea lo que devuelve.
Neo4j (grafo de conocimiento)	Sustituido. En GrafoCargaService se parchea neo4j_service con un driver y una sesión falsos que responden a las consultas y registran los MERGE ejecutados. No se abre ninguna conexión.
pgvector sobre Supabase (almacén vectorial)	Sustituido. En RecuperacionService se parchean _buscar_fragmentos y _coser_ventana; en AuditoriaService se parchea paper_ya_indexado. No se calcula ningún embedding real.
CrossRef / Unpaywall (verificación de DOI)	No se invocan. Las pruebas de doi_utils ejercitan únicamente la normalización local del identificador.
Persistencia del veredicto	Sustituida. Se parchea _persistir_veredicto, de modo que la emisión del veredicto se comprueba sin escritura alguna.
Extracción del PDF (PyMuPDF)	Real, pero local. PyMuPDF trabaja sobre el archivo sin acceso a red: es la única dependencia pesada que se ejecuta de verdad, por ser el objeto de prueba.
scikit-learn (métricas)	Real y local. Las pruebas de metricas_service verifican la aritmética del Kappa, el F1 y la matriz de confusión.

En consecuencia, la suite es determinista y reproducible: dos ejecuciones consecutivas sobre el mismo código producen idéntico resultado, sin coste de inferencia ni dependencia de la disponibilidad de servicios de terceros.

4. Entorno de ejecución

Elemento	Valor
Sistema operativo	Linux 6.6.87.2-microsoft-standard-WSL2 (WSL2 sobre Windows)
Intérprete	Python 3.11.15
Entorno virtual	backend/venv-wsl
Framework de pruebas	pytest 8.3.4
Complementos	pluggy 1.6.0, anyio 4.14.1
Directorio raíz	backend/
Archivo de configuración	backend/pytest.ini
Comando ejecutado	python -m pytest tests/unit -v
Acceso a red durante la ejecución	Ninguno

Los parámetros de recolección declarados en backend/pytest.ini son los siguientes:

```
[pytest]
testpaths = tests/unit tests/integration

python_files = test_*.py
python_classes = Test*
python_functions = test_*

addopts = -v --tb=short
```

Para la elaboración de este informe, la corrida se ejecutó además con un complemento de captura que envuelve las funciones del código bajo prueba y registra, para cada prueba, los argumentos

con los que se las invocó y el valor que devolvieron. El complemento solo observa: no altera el comportamiento del código ni el resultado de las pruebas, que es el mismo con y sin él.

5. Resumen de resultados

La totalidad de las 116 pruebas de la suite se ejecutó con resultado satisfactorio.

Métrica	Valor
Pruebas recolectadas	116
Pruebas superadas (PASSED)	116
Pruebas fallidas (FAILED)	0
Errores (ERROR)	0
Pruebas omitidas (SKIPPED)	0
Tasa de éxito	100 %
Tiempo total de ejecución	26,71 s
Advertencias emitidas	9 (deprecaciones internas de bibliotecas de terceros; no afectan al resultado)

El desglose por módulo es el siguiente:

Archivo de pruebas	Componente	Pruebas	Superadas	Fallidas
test_pdf_service.py	PDFExtractionService	19	19	0
test_extraccion_service.py	EntidadExtractionService	31	31	0
test_grafo_carga_service.py	GrafoCargaService	12	12	0
test_recuperacion_service.py	RecuperacionService	8	8	0
test_auditoria_service.py	AuditoriaService	14	14	0
test_metricas_service.py	metricas_service	12	12	0
test_doi_utils.py	doi_utils	12	12	0
test_texto.py	limpiar_fragmento	8	8	0
Total		116	116	0

La salida final del ejecutor fue:

```
===== 116 passed, 9 warnings in 26.71s =====
```

El grueso del tiempo de ejecución corresponde a la extracción del texto del PDF (5,75 s), que se realiza una sola vez por sesión y se comparte entre todas las pruebas. Ninguna prueba individual supera los 0,03 s de tiempo de cómputo propio.

6. Fixtures compartidas

Las fixtures se definen en tests/unit/conftest.py. La extracción del texto del PDF es costosa, de modo que se realiza una única vez por sesión y su resultado se comparte entre todas las pruebas que lo necesitan. Las fixtures derivadas (cuerpo sin anexos, sección de referencias) se obtienen de ese mismo texto.

Fixture	Ámbito	Descripción y valor en la ejecución
---------	--------	-------------------------------------

ruta_pdf	session	Copia el PDF real de la tesis a un directorio temporal, ya que la extracción deja un archivo .md de caché junto al original y el repositorio no debe alterarse. Si el PDF no está presente, las pruebas que dependen de él se omiten.
_extraccion	session	Ejecuta pdf_service.extraer_texto una única vez y devuelve la tupla (texto, número de páginas). Es el punto de coste de la suite: 5,75 s.
texto_tesis	session	Markdown completo del documento de prueba: 121 193 caracteres.
paginas_tesis	session	Número de páginas del documento de prueba: 67.
texto_cuerpo	session	Cuerpo del documento sin los anexos, tal como lo ve la detección de citas: 96 093 caracteres.
seccion_referencias	session	Texto de la sección bibliográfica, delimitada por su encabezado y el fin de sección: 34 635 caracteres.
svc	function	Instancia nueva de EntidadExtractionService para cada prueba, de modo que ninguna herede estado de otra.

El código de las fixtures compartidas es el siguiente:

```

"""
Fixtures compartidas por las pruebas unitarias.

El documento de prueba es un PDF real: la tesis del autor del proyecto. No se
usa texto inventado en ninguna prueba – todo lo que se afirma sobre citas,
referencias, secciones o DOIs se comprueba contra ese documento.

La extracción del PDF es cara (~7 s), así que se hace una sola vez por sesión
y se comparte. El PDF se copia a un directorio temporal porque extraer_texto
escribe un .md de caché junto al original, y el repositorio no debe ensuciarse.
"""
import shutil
from pathlib import Path

import pytest

from app.services.ingesta.pdf_service import pdf_service
from app.services.grafo.extraccion_service import EntidadExtractionService

PDF_TESIS = Path(__file__).parent / "Informe de tesis_AlcaldeLavadoMatias (1).pdf"

@pytest.fixture(scope="session")
def ruta_pdf(tmp_path_factory) -> Path:
    """Copia del PDF real en un temporal, para no dejar la caché .md en el repo."""
    if not PDF_TESIS.exists():
        pytest.skip(f"No se encontró el PDF de prueba: {PDF_TESIS.name}")
    destino = tmp_path_factory.mktemp("documento") / "tesis.pdf"
    shutil.copy(PDF_TESIS, destino)
    return destino

@pytest.fixture(scope="session")
def _extraccion(ruta_pdf) -> tuple[str, int]:
    return pdf_service.extraer_texto(ruta_pdf)

@pytest.fixture(scope="session")
def texto_tesis(_extraccion) -> str:
    """Markdown completo extraído de la tesis real."""

```

```

    return _extraccion[0]

@pytest.fixture(scope="session")
def paginas_tesis(_extraccion) -> int:
    return _extraccion[1]

@pytest.fixture(scope="session")
def texto_cuerpo(texto_tesis) -> str:
    """Cuerpo de la tesis sin los anexos (como lo ve la detección de citas)."""
    return EntidadExtractionService._truncar_antes_de_anexos(texto_tesis)

@pytest.fixture(scope="session")
def seccion_referencias(texto_tesis) -> str:
    """Texto de la sección de referencias bibliográficas de la tesis real."""
    svc = EntidadExtractionService()
    match = svc._encontrar_inicio_referencias(texto_tesis)
    assert match is not None, "La tesis de prueba debe tener sección de referencias"
    desde_refs = texto_tesis[match.start():]
    return desde_refs[: svc._encontrar_fin_seccion(desde_refs)]

@pytest.fixture
def svc() -> EntidadExtractionService:
    return EntidadExtractionService()

```

7. Detalle de las pruebas

Se documenta a continuación cada una de las 116 pruebas, agrupadas por módulo y por clase. De cada prueba se indica su descripción, su código fuente, la entrada con la que se ejerció el componente, la salida que este devolvió y el resultado de la comprobación. Los valores muy extensos (el texto completo del documento, el contenido de un PDF) se abrevian indicando su tamaño real.

7.1. PDFExtractionService

Archivo de pruebas	Módulo bajo prueba	Pruebas
tests/unit/test_pdf_service.py	app/services/ingesta/pdf_service.py	19

PDFExtractionService — validación, extracción de texto y detección de secciones.

Se ejecuta contra el PDF real de la tesis (67 páginas). Sin red ni servicios externos: PyMuPDF trabaja en local.

7.1.1. TestValidarPDF

validar_pdf rechaza lo que no debe entrar al pipeline.

test_pdf_real_es_valido

Descripción. El PDF real de la tesis supera la validación de entrada del pipeline sin lanzar excepción.

Código.

Entrada.

Salida.

Resultado. PASSED

Código.

Entrada.

Salida.

Resultado. PASSED

Código.

Entrada.

[illegible]

Salida.

```
lanza PDFNoProcessableError | codigo='ARCHIVO_DEMASIADO_GRANDE' | mensaje='El archivo pesa 10.0 MB y supera el límite de 10 MB.' | accion='Comprime el PDF o divídelo en partes más pequeñas.'
```

Resultado. PASSED

test_rechaza_contenido_sin_cabecera_pdf

Descripción. Un contenido sin la cabecera %PDF se rechaza con el código PDF_CORRUPTO.

Código.

```
def test_rechaza_contenido_sin_cabecera_pdf(self):
    with pytest.raises(PDFNoProcessableError) as exc:
        pdf_service.validar_pdf(b"esto no es un PDF", "tesis.pdf")
    assert exc.value.codigo == "PDF_CORRUPTO"
```

Entrada.

```
PDFExtractionService.validar_pdf(<17 bytes> b'esto no es un PDF'..., 'tesis.pdf')
```

Salida.

```
lanza PDFNoProcessableError | codigo='PDF_CORRUPTO' | mensaje='El archivo no tiene una estructura PDF válida.' | accion='Verifica que el archivo no esté dañado e intenta exportarlo nuevamente.'
```

Resultado. PASSED

7.1.2. TestExtraerTexto

extraer_texto sobre la tesis real.

test_devuelve_el_numero_real_de_paginas

Descripción. La extracción informa el número real de páginas del documento de prueba: 67.

Código.

```
def test_devuelve_el_numero_real_de_paginas(self, paginas_tesis):
    assert paginas_tesis == 67
```

Entrada.

```
Documento de prueba (PDF de la tesis), vía fixture paginas_tesis
```

Salida.

```
paginas_tesis = 67
```

Resultado. PASSED

test_extrae_el_cuerpo_completo_del_documento

Descripción. El texto extraído supera los 100 000 caracteres, lo que confirma que se procesó el documento completo y no solo sus primeras páginas.

Código.


```
def test_extrae_el_cuerpo_completo_del_documento(self, texto_tesis):
    assert len(texto_tesis) > 100_000
```

Entrada.

Documento de prueba (PDF de la tesis), vía fixture texto_tesis

Salida.

len(texto_tesis) = 121 193 caracteres (> 100 000)

Resultado. PASSED

test_el_texto_conserva_el_titulo_de_la_tesis

Descripción. El término «GraphRAG», presente en el título de la tesis, sobrevive a la conversión a markdown.

Código.

```
def test_el_texto_conserva_el_titulo_de_la_tesis(self, texto_tesis):
    assert "GraphRAG" in texto_tesis
```

Entrada.

texto_tesis (markdown extraído del documento de prueba)

Salida.

'GraphRAG' in texto_tesis = True (74 apariciones)

Resultado. PASSED

test_el_texto_no_arrastra_caracteres_invisibles

Descripción. El texto extraído no contiene caracteres invisibles del PDF (espacios de ancho cero, guiones suaves ni marcas de orden de bytes).

Código.

```
def test_el_texto_no_arrastra_caracteres_invisibles(self, texto_tesis):
    invisibles = ["", "", "", "", "-"]
    assert not any(ch in texto_tesis for ch in invisibles)
```

Entrada.

texto_tesis y la lista de caracteres invisibles ['\u200b', '\u200c', '\u200d', '\uffff', '\u00ad']

Salida.

Apariciones en texto_tesis: '\u200b': 0, '\u200c': 0, '\u200d': 0, '\uffff': 0, '\xad': 0
→
ninguno presente

Resultado. PASSED

test_segunda_llamada_usa_la_cache_en_disco

Descripción. La primera extracción deja un archivo .md de caché junto al PDF y una segunda llamada devuelve exactamente el mismo texto y número de páginas.

Código.

```
def test_segunda_llamada_usa_la_cache_en_disco(self, ruta_pdf, texto_tesis):
    cache = ruta_pdf.with_suffix(".md")
    assert cache.exists() # la extracción previa dejó la caché
    texto_2, paginas_2 = pdf_service.extraer_texto(ruta_pdf)
    assert texto_2 == texto_tesis
    assert paginas_2 == 67
```

Entrada.

```
PDFExtractionService.extraer_texto(PosixPath('/tmp/pytest-of-matias/pytest-
5/documento0/tesis.pdf
f'))
```

Salida.

```
('UNIVERSIDAD PRIVADA ANTONOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE
INGENIERÍA DE COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL
DE\n\nINGENIERO DE COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con
evaluación experta en detección de\n\ninconsistencias en citaciones bibliográficas de
tesis,
UPAO 2024-2025\n\nLínea de Investigación:\n\nRobótica, automatización avanzada y
sistemas
inteligentes.\n\nSub Línea de Investigación: Sistemas Inteligentes\n\nAutor: Alcalde
Lavado
Matias Felipe\n\n\n**Jurado
evaluador:**\n\nPresidente:\n\nSecretario:\n\nVocal:\n\n\nApellido
os, Nombres\n\nApellidos, Nombres\n\nAsesor:\n\nCueva Chavez, Walter\n\n\n[Códi ...
[124950
caracteres en total]
```

Resultado. PASSED

7.1.3. TestObtenerTOC

obtener_toc nunca rompe el pipeline: si no hay índice, devuelve lista vacía.

test_la_tesis_no_trae_indice_embebido

Descripción. El documento de prueba no incluye marcadores de índice embebidos y la función devuelve una lista vacía en lugar de fallar.

Código.

```
def test_la_tesis_no_trae_indice_embebido(self, ruta_pdf):
    assert pdf_service.obtener_toc(ruta_pdf) == []
```

Entrada.

```
PDFExtractionService.obtener_toc(PosixPath('/tmp/pytest-of-matias/pytest-
5/documento0/tesis.pdf'
))
```

Salida.

```
[]
```

Resultado. PASSED

test_ruta_inexistente_devuelve_lista_vacia

Descripción. Una ruta a un archivo inexistente devuelve una lista vacía sin interrumpir el pipeline.

Código.

```
def test_ruta_inexistente_devuelve_lista_vacia(self, tmp_path):
    assert pdf_service.obtener_toc(tmp_path / "no_existe.pdf") == []
```

Entrada.

```
PDFExtractionService.obtener_toc(PosixPath('/tmp/pytest-of-matias/pytest-5/test_ruta_inexistente_devuelve0/no_existe.pdf'))
```

Salida.

```
[]
```

Resultado. PASSED

7.1.4. TestDetectarSecciones

detectar_secciones sobre la estructura real de la tesis.

test_detecta_la_seccion_de_referencias

Descripción. El detector de secciones reconoce que el documento de prueba tiene sección de referencias bibliográficas.

Código.

```
def test_detecta_la_seccion_de_referencias(self, texto_tesis, paginas_tesis):
    estructura = pdf_service.detectar_secciones(texto_tesis, paginas_tesis,
"doc-1")
    assert estructura.tiene_seccion_referencias is True
```

Entrada.

```
PDFExtractionService.detectar_secciones(<cadena de 121 193 caracteres> 'UNIVERSIDAD PRIVADA
ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA DE COMPUTACIÓN
Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta '...', 67, 'doc-1')
```

Salida.

```
EstructuraDocumentoResponse(documento_id='doc-1', total_paginas=67,
secciones=[SeccionDetectada(tipo=<TipoSeccion.REFERENCIAS: 'referencias'>,
titulo_detectado='7. Referencias bibliográficas', pagina_inicio=52, pagina_fin=67,
tiene_referencias=True)], tiene_seccion_referencias=True, advertencia=None)
```

Resultado. PASSED

test_el_titulo_detectado_es_el_de_la_tesis

Descripción. Se detecta exactamente una sección de tipo REFERENCIAS, su título contiene «Referencias bibliográficas» y queda marcada como portadora de referencias.

Código.

```
def test_el_titulo_detectado_es_el_de_la_tesis(self, texto_tesis, paginas_tesis):
    estructura = pdf_service.detectar_secciones(texto_tesis, paginas_tesis,
"doc-1")
    refs = [s for s in estructura.secciones if s.tipo ==
TipoSeccion.REFERENCIAS]
    assert len(refs) == 1
```

```
assert "Referencias bibliográficas" in refs[0].titulo_detectado
assert refs[0].tiene_referencias is True
```

Entrada.

```
PDFExtractionService.detectar_secciones(<cadena de 121 193 caracteres> 'UNIVERSIDAD
PRIVADA
ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA DE
COMPUTACIÓN
Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE COMPUTACIÓN Y
SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta '..., 67, 'doc-
1')
```

Salida.

```
EstructuraDocumentoResponse(documento_id='doc-1', total_paginas=67,
secciones=[SeccionDetectada(tipo=<TipoSeccion.REFERENCIAS: 'referencias'>,
titulo_detectado='7. Referencias bibliográficas', pagina_inicio=52, pagina_fin=67,
tiene_referencias=True)], tiene_seccion_referencias=True, advertencia=None)
```

Resultado. PASSED

test_sin_referencias_emite_advertencia

Descripción. Un documento sin bibliografía se marca como carente de sección de referencias y la estructura devuelta incluye una advertencia.

Código.

```
def test_sin_referencias_emite_advertencia(self, paginas_tesis):
    estructura = pdf_service.detectar_secciones(
        "# Introducción\n\nTexto sin bibliografía.", paginas_tesis, "doc-1"
    )
    assert estructura.tiene_seccion_referencias is False
    assert estructura.advertencia is not None
```

Entrada.

```
PDFExtractionService.detectar_secciones('# Introducción\n\nTexto sin bibliografía.', 67,
'doc-1')
```

Salida.

```
EstructuraDocumentoResponse(documento_id='doc-1', total_paginas=67,
secciones=[SeccionDetectada(tipo=<TipoSeccion.INTRODUCCION: 'introduccion'>,
titulo_detectado='Introducción', pagina_inicio=1, pagina_fin=67,
tiene_referencias=False)],
tiene_seccion_referencias=False, advertencia='No se detectó una sección de Referencias
o
Bibliografía. La auditoría puede estar incompleta.')
```

Resultado. PASSED

test_las_paginas_de_la_seccion_estan_dentro_del_documento

Descripción. Toda sección detectada tiene página de inicio y de fin dentro del rango de páginas del documento, y su inicio nunca es posterior a su fin.

Código.

```
def test_las_paginas_de_la_seccion_estan_dentro_del_documento(self, texto_tesis,
paginas_tesis):
    estructura = pdf_service.detectar_secciones(texto_tesis, paginas_tesis,
"doc-1")
```

```

for seccion in estructura.secciones:
    assert 1 <= seccion.pagina_inicio <= paginas_tesis
    assert seccion.pagina_inicio <= seccion.pagina_fin <= paginas_tesis

```

Entrada.

```

PDFExtractionService.detectar_secciones(<cadena de 121 193 caracteres> 'UNIVERSIDAD
PRIVADA
ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA DE
COMPUTACIÓN
Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE COMPUTACIÓN Y
SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta '..., 67, 'doc-
1')

```

Salida.

```

EstructuraDocumentoResponse(documento_id='doc-1', total_paginas=67,
secciones=[SeccionDetectada(tipo=<TipoSeccion.REFERENCIAS: 'referencias'>,
titulo_detectado='7. Referencias bibliográficas', pagina_inicio=52, pagina_fin=67,
tiene_referencias=True)], tiene_seccion_referencias=True, advertencia=None)

```

Resultado. PASSED

7.1.5. TestNormalizarYClasificar

Helpers de clasificación, con los encabezados tal como salen del markdown.

test_normalizar_quita_almohadillas_y_negritas

Descripción. La normalización de títulos elimina las almohadillas de encabezado y los asteriscos de negrita del markdown.

Código.

```

def test_normalizar_quita_almohadillas_y_negritas(self):
    assert pdf_service._normalizar_titulo("## **7. Referencias**") == "7.
Referencias"

```

Entrada.

```

PDFExtractionService._normalizar_titulo('## **7. Referencias**')

```

Salida.

```

'7. Referencias'

```

Resultado. PASSED

test_clasifica_el_encabezado_real_de_la_tesis

Descripción. El encabezado real «7. Referencias bibliográficas» se clasifica como sección de tipo REFERENCIAS.

Código.

```

def test_clasifica_el_encabezado_real_de_la_tesis(self):
    assert pdf_service._clasificar_seccion("7. Referencias bibliográficas") ==
TipoSeccion.REFERENCIAS

```

Entrada.

```

PDFExtractionService._clasificar_seccion('7. Referencias bibliográficas')

```

Salida.

```
<TipoSeccion.REFERENCIAS: 'referencias'>
```

Resultado. PASSED

test_clasifica_introduccion

Descripción. El encabezado «Introducción» se clasifica como sección de tipo INTRODUCCION.

Código.

```
def test_clasifica_introduccion(self):
    assert pdf_service._clasificar_seccion("Introducción") ==
    TipoSeccion.INTRODUCCION
```

Entrada.

```
PDFExtractionService._clasificar_seccion('Introducción')
```

Salida.

```
<TipoSeccion.INTRODUCCION: 'introduccion'>
```

Resultado. PASSED

test_un_titulo_cualquiera_queda_como_desconocido

Descripción. Un encabezado no contemplado por la taxonomía se clasifica como DESCONOCIDO en lugar de forzar una categoría.

Código.

```
def test_un_titulo_cualquiera_queda_como_desconocido(self):
    assert pdf_service._clasificar_seccion("Agradecimientos") ==
    TipoSeccion.DESCONOCIDO
```

Entrada.

```
PDFExtractionService._clasificar_seccion('Agradecimientos')
```

Salida.

```
<TipoSeccion.DESCONOCIDO: 'desconocido'>
```

Resultado. PASSED

7.2. EntidadExtractionService

Archivo de pruebas	Módulo bajo prueba	Pruebas
tests/unit/test_extraccion_service.py	app/services/grafos/extraccion_service.py	31

EntidadExtractionService — detección de citas y delimitación de secciones.

Todo se prueba sobre el texto real de la tesis. Las funciones que llaman al LLM (extraer_citas / extraer_referencias) no se tocan aquí: lo que se prueba es la lógica determinista que decide QUÉ texto se le manda al LLM y cómo se limpia lo que devuelve.

7.2.1. TestDetectarCitasRegex

Prefiltro por regex: es lo que ancla la detección antes del LLM.

test_encuentra_citas_en_la_tesis_real

Descripción. El prefiltro por expresión regular detecta al menos treinta citas en el texto real de la tesis.

Código.

```
def test_encuentra_citas_en_la_tesis_real(self, svc, texto_tesis):
    citas = svc._detectar_citas_regex(texto_tesis)
    assert len(citas) >= 30
```

Entrada.

```
EntidadExtractionService._detectar_citas_regex(<cadena de 121 193 caracteres>
'UNIVERSIDAD
PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA
DE
COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE
COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta
'...)
```

Salida.

```
['Sarmah et al. (2024)', '(Kacena et al., 2024)', 'Wadden\n\net al. (2020)', '(Bakker et
al.,
2023)', '(Fan et al., 2024)', 'Hu et al. (2024)', 'Hang et al. (2025)', '(George et
al.,\n\n2025)', 'Dunford et al. (2024)', 'Wadden et al.\n\n(2020)', 'Es et
al.\n\n(2025)',
'Magesh et al. (2025)', '(Zhao et al., 2026)', 'Zapata et al. (2023)', '(Wakeling et
al.,
2025)', 'Arévalo et al.\n\n(2021)', '(Edge et al., 2024)', '(Klesel y Wittmann,
2025)',
'Antal y Buza (2025)', 'Chen et al. (2025)', 'Peng et al. (2024)', '(Bakker\n\net al.,
2023)', 'Thakur et al. (2025)', 'Scaffidi et al. (2025)', 'Nagori et al. (2025)',
'(Mars,
2022)', '(Liu et al., 2023)', 'Walters y Wilder (2023)', '(Hang\n\net al., 2
... [930
caracteres en total]
```

Resultado. PASSED

test_detecta_una_parentetica_con_et_al

Descripción. Se detecta la cita parentética con abreviatura et al. «(Edge et al., 2024)», presente en el documento de prueba.

Código.

```
def test_detecta_una_parentetica_con_et_al(self, svc, texto_tesis):
    citas = svc._detectar_citas_regex(texto_tesis)
    assert "(Edge et al., 2024)" in citas
```

Entrada.

```
EntidadExtractionService._detectar_citas_regex(<cadena de 121 193 caracteres>
'UNIVERSIDAD
PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA
DE
COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE
COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta
'...)
```

Salida.

```
[ 'Sarmah et al. (2024)', '(Kacena et al., 2024)', 'Wadden\n\net al. (2020)', '(Bakker et al., 2023)', '(Fan et al., 2024)', 'Hu et al. (2024)', 'Hang et al. (2025)', '(George et al.,\n\n2025)', 'Dunford et al. (2024)', 'Wadden et al.\n\n(2020)', 'Es et al.\n\n(2025)', 'Magesh et al. (2025)', '(Zhao et al., 2026)', 'Zapata et al. (2023)', '(Wakeling et al., 2025)', 'Arévalo et al.\n\n(2021)', '(Edge et al., 2024)', '(Klesel y Wittmann, 2025)', 'Antal y Buza (2025)', 'Chen et al. (2025)', 'Peng et al. (2024)', '(Bakker\n\net al., 2023)', 'Thakur et al. (2025)', 'Scaffidi et al. (2025)', 'Nagori et al. (2025)', '(Mars, 2022)', '(Liu et al., 2023)', 'Walters y Wilder (2023)', '(Hang\n\net al., 2 ... [930 caracteres en total]
```

Resultado. PASSED

test_detecta_una_parentetica_de_dos_autores_con_y

Descripción. Se detecta la cita parentética de dos autores unidos por la conjunción «y»: «(Klesel y Wittmann, 2025)».

Código.

```
def test_detecta_una_parentetica_de_dos_autores_con_y(self, svc, texto_tesis):
    citas = svc._detectar_citas_regex(texto_tesis)
    assert "(Klesel y Wittmann, 2025)" in citas
```

Entrada.

```
EntidadExtractionService._detectar_citas_regex(<cadena de 121 193 caracteres>
'UNIVERSIDAD
PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA
DE
COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE
COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta
'...)
```

Salida.

```
[ 'Sarmah et al. (2024)', '(Kacena et al., 2024)', 'Wadden\n\net al. (2020)', '(Bakker et al., 2023)', '(Fan et al., 2024)', 'Hu et al. (2024)', 'Hang et al. (2025)', '(George et al.,\n\n2025)', 'Dunford et al. (2024)', 'Wadden et al.\n\n(2020)', 'Es et al.\n\n(2025)', 'Magesh et al. (2025)', '(Zhao et al., 2026)', 'Zapata et al. (2023)', '(Wakeling et al., 2025)', 'Arévalo et al.\n\n(2021)', '(Edge et al., 2024)', '(Klesel y Wittmann, 2025)', 'Antal y Buza (2025)', 'Chen et al. (2025)', 'Peng et al. (2024)', '(Bakker\n\net al., 2023)', 'Thakur et al. (2025)', 'Scaffidi et al. (2025)', 'Nagori et al. (2025)', '(Mars, 2022)', '(Liu et al., 2023)', 'Walters y Wilder (2023)', '(Hang\n\net al., 2 ... [930 caracteres en total]
```

Resultado. PASSED

test_todas_las_citas_llevan_un_anio_valido

Descripción. Toda cita detectada contiene un año de cuatro dígitos válido, lo que descarta paréntesis que no son citas.

Código.


```
def test_todas_las_citas_llevan_un_anio_valido(self, svc, texto_tesis):
    citas = svc._detectar_citas_regex(texto_tesis)
    assert all(re.search(r"\b(19|20)\d{2}\b", c) for c in citas)
```

Entrada.

```
EntidadExtractionService._detectar_citas_regex(<cadena de 121 193 caracteres>
'UNIVERSIDAD
  PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA
DE
  COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE
  COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta
'...)
```

Salida.

```
['Sarmah et al. (2024)', '(Kacena et al., 2024)', 'Wadden\n\net al. (2020)', '(Bakker et
al.,
  2023)', '(Fan et al., 2024)', 'Hu et al. (2024)', 'Hang et al. (2025)', '(George et
al.,\n\n2025)', 'Dunford et al. (2024)', 'Wadden et al.\n\n(2020)', 'Es et
al.\n\n(2025)',
'Magesh et al. (2025)', '(Zhao et al., 2026)', 'Zapata et al. (2023)', '(Wakeling et
al.,
  2025)', 'Arévalo et al.\n\n(2021)', '(Edge et al., 2024)', '(Klesel y Wittmann,
2025)',
'Antal y Buza (2025)', 'Chen et al. (2025)', 'Peng et al. (2024)', '(Bakker\n\net al.,
2023)', 'Thakur et al. (2025)', 'Scaffidi et al. (2025)', 'Nagori et al. (2025)',
'(Mars,
  2022)', '(Liu et al., 2023)', 'Walters y Wilder (2023)', '(Hang\n\net al., 2
... [930
caracteres en total]
```

Resultado. PASSED

test_no_devuelve_subcoincidencias_de_otra_cita

Descripción. Ninguna cita detectada está contenida dentro de otra cita detectada, lo que garantiza que no se emiten fragmentos duplicados.

Código.

```
def test_no_devuelve_subcoincidencias_de_otra_cita(self, svc, texto_tesis):
    citas = svc._detectar_citas_regex(texto_tesis)
    for cita in citas:
        assert not any(cita != otra and cita in otra for otra in citas)
```

Entrada.

```
EntidadExtractionService._detectar_citas_regex(<cadena de 121 193 caracteres>
'UNIVERSIDAD
  PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA
DE
  COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE
  COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta
'...)
```

Salida.

```
['Sarmah et al. (2024)', '(Kacena et al., 2024)', 'Wadden\n\net al. (2020)', '(Bakker et
al.,
  2023)', '(Fan et al., 2024)', 'Hu et al. (2024)', 'Hang et al. (2025)', '(George et
al.,\n\n2025)', 'Dunford et al. (2024)', 'Wadden et al.\n\n(2020)', 'Es et
al.\n\n(2025)',
'Magesh et al. (2025)', '(Zhao et al., 2026)', 'Zapata et al. (2023)', '(Wakeling et
al.,
```

```
2025)', 'Arévalo et al.\n\n(2021)', '(Edge et al., 2024)', '(Klesel y Wittmann,
2025)',
'Antal y Buza (2025)', 'Chen et al. (2025)', 'Peng et al. (2024)', '(Bakker\n\net al.,
2023)', 'Thakur et al. (2025)', 'Scaffidi et al. (2025)', 'Nagori et al. (2025)',
'(Mars,
2022)', '(Liu et al., 2023)', 'Walters y Wilder (2023)', '(Hang\n\net al., 2 ... [930
caracteres en total]
```

Resultado. PASSED

test_un_texto_sin_citas_no_devuelve_nada

Descripción. Un párrafo sin ninguna cita APA no produce detecciones.

Código.

```
def test_un_texto_sin_citas_no_devuelve_nada(self, svc):
    assert svc._detectar_citas_regex("Un párrafo cualquiera sin ninguna cita
APA.") == []
```

Entrada.

```
EntidadExtractionService._detectar_citas_regex('Un párrafo cualquiera sin ninguna cita
APA.')
```

Salida.

```
[]
```

Resultado. PASSED

7.2.2. TestSeccionDeReferencias

Delimitación de la bibliografía dentro del documento.

test_encuentra_el_encabezado_de_referencias

Descripción. Se localiza el encabezado de la sección bibliográfica del documento de prueba.

Código.

```
def test_encuentra_el_encabezado_de_referencias(self, svc, texto_tesis):
    match = svc._encontrar_inicio_referencias(texto_tesis)
    assert match is not None
    assert "Referencias bibliográficas" in match.group()
```

Entrada.

```
EntidadExtractionService._encontrar_inicio_referencias(<cadena de 121 193 caracteres>
'UNIVERSIDAD PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO
DE
INGENIERÍA DE COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL
DE\n\nINGENIERO DE COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con
evaluación experta '...)
```

Salida.

```
<re.Match object; span=(86558, 86590), match='\n7. Referencias bibliográficas\n\n'>
```

Resultado. PASSED

test_el_encabezado_esta_en_la_parte_final_del_documento

Descripción. El encabezado de referencias aparece en la segunda mitad del documento, lo que descarta una coincidencia con la mención del índice.

Código.

```
def test_el_encabezado_esta_en_la_parte_final_del_documento(self, svc, texto_tesis):
    match = svc._encontrar_inicio_referencias(texto_tesis)
    assert match.start() > len(texto_tesis) * 0.5
```

Entrada.

```
EntidadExtractionService._encontrar_inicio_referencias(<cadena de 121 193 caracteres>
'UNIVERSIDAD PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO
DE
INGENIERÍA DE COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL
DE\n\nINGENIERO DE COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con
evaluación experta '...)
```

Salida.

```
<re.Match object; span=(86558, 86590), match='\n7. Referencias bibliográficas\n\n'>
```

Resultado. PASSED

test_un_texto_sin_bibliografia_no_devuelve_encabezado

Descripción. Un texto sin bibliografía no produce ninguna coincidencia de encabezado.

Código.

```
def test_un_texto_sin_bibliografia_no_devuelve_encabezado(self, svc):
    assert svc._encontrar_inicio_referencias("# Introducción\n\nTexto del
cuerpo.") is None
```

Entrada.

```
EntidadExtractionService._encontrar_inicio_referencias('# Introducción\n\nTexto del
cuerpo.')
```

Salida.

```
None
```

Resultado. PASSED

test_la_seccion_contiene_las_entradas_bibliograficas

Descripción. La sección delimitada contiene una entrada bibliográfica real del documento y supera los 10 000 caracteres.

Código.

```
def test_la_seccion_contiene_las_entradas_bibliograficas(self, seccion_referencias):
    assert "Ansari, S. (2026)" in seccion_referencias
    assert len(seccion_referencias) > 10_000
```

Entrada.

```
seccion_referencias (sección bibliográfica delimitada del documento de prueba)
```

Salida.

```
'Ansari, S. (2026)' in seccion_referencias = True; len(seccion_referencias) = 34 635
caracteres
(> 10 000)
```

Resultado. PASSED

test_la_seccion_contiene_los_doi_de_las_fuentes

Descripción. La sección delimitada contiene al menos treinta DOI, que son las claves con las que después se verifican las fuentes.

Código.

```
def test_la_seccion_contiene_los_doi_de_las_fuentes(self, seccion_referencias):
    dois = re.findall(r"10\.\d{4,9}/\S+", seccion_referencias)
    assert len(dois) >= 30
```

Entrada.

seccion_referencias y el patrón `r'10\.\d{4,9}/\S+'`

Salida.

```
DOI encontrados = 37 (>= 30). Primeros: 10.48550/arXiv.2602.05930, 10.1007/s44427-025-
00006-3_,
10.3390/educsci11040173
```

Resultado. PASSED

7.2.3. TestTruncarAntesDeAnexos

Los anexos no se auditan: se recortan antes de detectar nada.

test_recorta_los_anexos_de_la_tesis

Descripción. El truncado previo a los anexos produce un cuerpo estrictamente más corto que el documento completo.

Código.

```
def test_recorta_los_anexos_de_la_tesis(self, svc, texto_tesis, texto_cuerpo):
    assert len(texto_cuerpo) < len(texto_tesis)
```

Entrada.

texto_tesis y texto_cuerpo (resultado de `_truncar_antes_de_anexos`)

Salida.

```
len(texto_cuerpo) = 96 093 < len(texto_tesis) = 121 193; se recortan 25 100 caracteres de
anexos
```

Resultado. PASSED

test_el_cuerpo_es_un_prefijo_exacto_del_documento

Descripción. El cuerpo resultante es un prefijo literal del documento: el truncado corta, no reescribe.

Código.

```
def test_el_cuerpo_es_un_prefijo_exacto_del_documento(self, texto_tesis,
    texto_cuerpo):
    assert texto_tesis.startswith(texto_cuerpo)
```

Entrada.

texto_tesis y texto_cuerpo

Salida.

texto_tesis.startswith(texto_cuerpo) = True

Resultado. PASSED

test_conserva_la_bibliografia_que_precede_a_los_anexos

Descripción. El truncado conserva la sección de referencias bibliográficas, que precede a los anexos.

Código.

```
def test_conserva_la_bibliografia_que_precede_a_los_anexos(self, texto_cuerpo):
    assert "Referencias bibliográficas" in texto_cuerpo
```

Entrada.

texto_cuerpo (documento sin anexos)

Salida.

'Referencias bibliográficas' in texto_cuerpo = True

Resultado. PASSED

test_un_texto_sin_anexos_no_se_recorta

Descripción. Un texto sin anexos se devuelve intacto.

Código.

```
def test_un_texto_sin_anexos_no_se_recorta(self, svc):
    texto = "# Introducción\n\nCuerpo del documento sin apéndices."
    assert svc._truncar_antes_de_anexos(texto) == texto
```

Entrada.

```
EntidadExtractionService._truncar_antes_de_anexos('# Introducción\n\nCuerpo del documento
sin
apéndices.')
```

Salida.

'# Introducción\n\nCuerpo del documento sin apéndices.'

Resultado. PASSED

7.2.4. TestIndiceYSeccionesNoCitables

El índice del propio texto sirve de mapa para descartar secciones.

test_parsea_el_indice_de_la_tesis

Descripción. Del índice del propio documento se extraen al menos cuarenta entradas de título.

Código.

```
def test_parsea_el_indice_de_la_tesis(self, svc, texto_tesis):
    entradas = svc._titulos_desde_indice(texto_tesis)
    assert len(entradas) >= 40
```

Entrada.

```
EntidadExtractionService._titulos_desde_indice(<cadena de 121 193 caracteres>
'UNIVERSIDAD
  PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA
DE
  COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE
  COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta
'...)
```

Salida.

```
[(1, '1. Introducción'), (2, '1.1. Contexto y antecedentes'), (2, '1.2. Descripción y
  justificación del estudio'), (2, '1.3. Organización del documento'), (1, '2.
Planteamiento
  del problema de investigación'), (2, '2.1. Descripción y delimitación del problema'),
(3,
  '2.1.1. Formulación del problema'), (3, '2.1.2. Problema central del estudio'), (2,
'2.2.
  Objetivos de la investigación'), (3, '2.2.1. Objetivo general'), (3, '2.2.2. Objetivos
  específicos'), (2, '2.3. Importancia del estudio'), (2, '2.4. Justificación de la
  investigación'), (3, '2.4.1. Justificación técnica'), (3, '2.4.2. Justificación
práctica'),
  (3, '2.4.3. Justificación metodológica'), (3, '2.4.4. Justificación social'), (2, ...
[1907
  caracteres en total]
```

Resultado. PASSED

test_el_nivel_se_infiere_de_la_numeracion

Descripción. El nivel jerárquico de cada título se deduce de su numeración: «1. Introducción» es nivel 1 y «1.1. Contexto y antecedentes» es nivel 2.

Código.

```
def test_el_nivel_se_infiere_de_la_numeracion(self, svc, texto_tesis):
    entradas = dict((titulo, nivel) for nivel, titulo in
    svc._titulos_desde_indice(texto_tesis))
    assert entradas["1. Introducción"] == 1
    assert entradas["1.1. Contexto y antecedentes"] == 2
```

Entrada.

```
EntidadExtractionService._titulos_desde_indice(<cadena de 121 193 caracteres>
'UNIVERSIDAD
  PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA
DE
  COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE
  COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta
'...)
```

Salida.

```
[(1, '1. Introducción'), (2, '1.1. Contexto y antecedentes'), (2, '1.2. Descripción y
  justificación del estudio'), (2, '1.3. Organización del documento'), (1, '2.
Planteamiento
  del problema de investigación'), (2, '2.1. Descripción y delimitación del problema'),
```

```
(3,
 '2.1.1. Formulación del problema'), (3, '2.1.2. Problema central del estudio'), (2,
 '2.2.
  Objetivos de la investigación'), (3, '2.2.1. Objetivo general'), (3, '2.2.2. Objetivos
  específicos'), (2, '2.3. Importancia del estudio'), (2, '2.4. Justificación de la
  investigación'), (3, '2.4.1. Justificación técnica'), (3, '2.4.2. Justificación
  práctica'),
  (3, '2.4.3. Justificación metodológica'), (3, '2.4.4. Justificación social'), (2, ...
 [1907
  caracteres en total]
```

Resultado. PASSED

test_descarta_el_marco_metodologico

Descripción. El recorte de secciones no citables elimina texto del documento, entre él el marco metodológico.

Código.

```
def test_descarta_el_marco_metodologico(self, svc, texto_tesis):
    recortado = svc._recortar_secciones_no_citables(texto_tesis, None)
    assert len(recortado) < len(texto_tesis)
```

Entrada.

```
EntidadExtractionService._recortar_secciones_no_citables(<cadena de 121 193 caracteres>
 'UNIVERSIDAD PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO
 DE
 INGENIERÍA DE COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL
 DE\n\nINGENIERO DE COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con
 evaluación experta '...', None)
```

Salida.

```
<cadena de 101 305 caracteres> 'UNIVERSIDAD PRIVADA ANTENOR ORREGO\n\nFACULTAD DE
 INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA DE COMPUTACIÓN Y SISTEMAS\n\nTESIS
 PARA
 OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE COMPUTACIÓN Y SISTEMAS\n\nConcordancia
 de un
 sistema GraphRAG con evaluación experta '...
```

Resultado. PASSED

test_sin_indice_el_texto_se_devuelve_intacto

Descripción. Un texto sin tabla de contenido se devuelve sin recortar, ya que no hay mapa de secciones que aplicar.

Código.

```
def test_sin_indice_el_texto_se_devuelve_intacto(self, svc):
    texto = "# Introducción\n\nCuerpo sin tabla de contenido."
    assert svc._recortar_secciones_no_citables(texto, None) == texto
```

Entrada.

```
EntidadExtractionService._recortar_secciones_no_citables('# Introducción\n\nCuerpo sin
 tabla de
 contenido.', None)
```

Salida.

```
'# Introducción\n\nCuerpo sin tabla de contenido.'
```

Resultado. PASSED

7.2.5. TestDividirEnBloques

El cuerpo se trocea para caber en la ventana del LLM.

test_trocea_el_cuerpo_de_la_tesis

Descripción. El cuerpo del documento se divide en más de un bloque para caber en la ventana de contexto del LLM.

Código.

```
def test_trocea_el_cuerpo_de_la_tesis(self, svc, texto_cuerpo):
    bloques = svc._dividir_en_bloques(texto_cuerpo)
    assert len(bloques) > 1
```

Entrada.

```
EntidadExtractionService._dividir_en_bloques(<cadena de 96 093 caracteres> 'UNIVERSIDAD
PRIVADA
    ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA DE
COMPUTACIÓN
    Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE COMPUTACIÓN Y
SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta '...)
```

Salida.

```
['UNIVERSIDAD PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE
INGENIERÍA DE COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL
DE\n\nINGENIERO DE COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con
evaluación experta en detección de\n\ninconsistencias en citaciones bibliográficas de
tesis,
    UPAO 2024-2025\n\nLínea de Investigación:\n\nRobótica, automatización avanzada y
sistemas
    inteligentes.\n\nSub Línea de Investigación: Sistemas Inteligentes\n\nAutor: Alcalde
Lavado
    Matias Felipe\n\n**Jurado
evaluador:**\n\nPresidente:\n\nSecretario:\n\nVocal:\n\nApellidos,
    Nombres\n\nApellidos, Nombres\n\nAsesor:\n\nCueva Chavez, Walter\n\n[Código Orc ...
[99007
    caracteres en total]
```

Resultado. PASSED

test_ningun_bloque_supera_el_maximo

Descripción. Ningún bloque generado supera los 7000 caracteres.

Código.

```
def test_ningun_bloque_supera_el_maximo(self, svc, texto_cuerpo):
    bloques = svc._dividir_en_bloques(texto_cuerpo)
    assert all(len(b) <= 7000 for b in bloques)
```

Entrada.

```
EntidadExtractionService._dividir_en_bloques(<cadena de 96 093 caracteres> 'UNIVERSIDAD
PRIVADA
    ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA DE
COMPUTACIÓN
    Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE COMPUTACIÓN Y
SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta '...)
```


Salida.

```
['UNIVERSIDAD PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE
INGENIERÍA DE COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL
DE\n\nINGENIERO DE COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con
evaluación experta en detección de\n\ninconsistencias en citas bibliográficas de
tesis,
UPAO 2024-2025\n\nLínea de Investigación:\n\nRobótica, automatización avanzada y
sistemas
inteligentes.\n\nSub Línea de Investigación: Sistemas Inteligentes\n\nAutor: Alcalde
Lavado
Matias Felipe\n\n**Jurado
evaluador:**\n\nPresidente:\n\nSecretario:\n\nVocal:\n\nApellidos,
Nombres\n\nApellidos, Nombres\n\nAsesor:\n\nCueva Chavez, Walter\n\n[Código Orc ...
[99007
caracteres en total]
```

Resultado. PASSED

test_no_se_generan_bloques_vacios

Descripción. Ningún bloque generado está vacío o compuesto solo por espacios.

Código.

```
def test_no_se_generan_bloques_vacios(self, svc, texto_cuerpo):
    bloques = svc._dividir_en_bloques(texto_cuerpo)
    assert all(b.strip() for b in bloques)
```

Entrada.

```
EntidadExtractionService._dividir_en_bloques(<cadena de 96 093 caracteres> 'UNIVERSIDAD
PRIVADA
ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA DE
COMPUTACIÓN
Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE COMPUTACIÓN Y
SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta '...)
```

Salida.

```
['UNIVERSIDAD PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE
INGENIERÍA DE COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL
DE\n\nINGENIERO DE COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con
evaluación experta en detección de\n\ninconsistencias en citas bibliográficas de
tesis,
UPAO 2024-2025\n\nLínea de Investigación:\n\nRobótica, automatización avanzada y
sistemas
inteligentes.\n\nSub Línea de Investigación: Sistemas Inteligentes\n\nAutor: Alcalde
Lavado
Matias Felipe\n\n**Jurado
evaluador:**\n\nPresidente:\n\nSecretario:\n\nVocal:\n\nApellidos,
Nombres\n\nApellidos, Nombres\n\nAsesor:\n\nCueva Chavez, Walter\n\n[Código Orc ...
[99007
caracteres en total]
```

Resultado. PASSED

test_las_citas_sobreviven_al_troceado

Descripción. Una cita real del documento sigue íntegra dentro de algún bloque tras el troceado: la división no parte las citas.

Código.

```
def test_las_citas_sobreviven_al_troceado(self, svc, texto_cuerpo):
    bloques = svc._dividir_en_bloques(texto_cuerpo)
    assert any("(Edge et al., 2024)" in b for b in bloques)
```

Entrada.

```
EntidadExtractionService._dividir_en_bloques(<cadena de 96 093 caracteres> 'UNIVERSIDAD
PRIVADA
ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA DE
COMPUTACIÓN
Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE COMPUTACIÓN Y
SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta '...)
```

Salida.

```
[ 'UNIVERSIDAD PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE
INGENIERÍA DE COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL
DE\n\nINGENIERO DE COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con
evaluación experta en detección de\n\ninconsistencias en citaciones bibliográficas de
tesis,
UPAO 2024-2025\n\nLínea de Investigación:\n\nRobótica, automatización avanzada y
sistemas
inteligentes.\n\nSub Línea de Investigación: Sistemas Inteligentes\n\nAutor: Alcalde
Lavado
Matias Felipe\n\n**Jurado
evaluador:**\n\nPresidente:\n\nSecretario:\n\nVocal:\n\nApellidos,
Nombres\n\nApellidos, Nombres\n\nAsesor:\n\nCueva Chavez, Walter\n\n[Código Orc ...
[99007
caracteres en total]
```

Resultado. PASSED

7.2.6. TestDeduplicarCitas

La misma ocurrencia extraída dos veces se colapsa; dos ocurrencias reales no.

test_colapsa_fragmentos_contenidos_y_conserva_el_mas_corto

Descripción. Dos extracciones de la misma cita cuyos fragmentos uno contiene al otro se colapsan en una sola, conservando el fragmento más corto.

Código.

```
def test_colapsa_fragmentos_contenidos_y_conserva_el_mas_corto(self, svc):
    corto = "GraphRAG mejora la recuperación (Edge et al., 2024)."
    largo = "En trabajos recientes se observa que GraphRAG mejora la
recuperación (Edge et al., 2024)."
    dedup = svc._deduplicar_citas([
        self._cita("(Edge et al., 2024)", largo, "c1"),
        self._cita("(Edge et al., 2024)", corto, "c2"),
    ])
    assert len(dedup) == 1
    assert dedup[0].fragmento_oracion == corto
```

Entrada.

```
EntidadExtractionService._deduplicar_citas([CitaEnTexto(cita_id='c1', texto_cita='(Edge
et al.,
2024)', tipo=<TipoCita.PARENTETICA: 'parentetica'>, pagina=1, fragmento_oracion='En
trabajos
recientes se observa que GraphRAG mejora la recuperación (Edge et al., 2024).',
referencia_id=None), CitaEnTexto(cita_id='c2', texto_cita='(Edge et al., 2024)',
tipo=<TipoCita.PARENTETICA: 'parentetica'>, pagina=1, fragmento_oracion='GraphRAG
```

```
mejora la
recuperación (Edge et al., 2024).', referencia_id=None)])
```

Salida.

```
[CitaEnTexto(cita_id='c2', texto_cita='(Edge et al., 2024)', tipo=<TipoCita.PARENTETICA:
'parentetica'>, pagina=1, fragmento_oracion='GraphRAG mejora la recuperación (Edge et
al.,
2024).', referencia_id=None)]
```

Resultado. PASSED

test_conserva_la_misma_cita_en_parrafos_distintos

Descripción. La misma cita aparecida en dos párrafos distintos se conserva como dos ocurrencias: no se deduplica por texto de cita.

Código.

```
def test_conserva_la_misma_cita_en_parrafos_distintos(self, svc):
    dedup = svc._deduplicar_citas([
        self._cita("(Edge et al., 2024)", "El grafo estructura el corpus.",
        "c1"),
        self._cita("(Edge et al., 2024)", "La evaluación se hizo sobre preguntas
globales.", "c2"),
    ])
    assert len(dedup) == 2
```

Entrada.

```
EntidadExtractionService._deduplicar_citas([CitaEnTexto(cita_id='c1', texto_cita='(Edge
et al.,
2024)', tipo=<TipoCita.PARENTETICA: 'parentetica'>, pagina=1, fragmento_oracion='El
grafo
estructura el corpus.', referencia_id=None), CitaEnTexto(cita_id='c2',
texto_cita='(Edge et
al., 2024)', tipo=<TipoCita.PARENTETICA: 'parentetica'>, pagina=1,
fragmento_oracion='La
evaluación se hizo sobre preguntas globales.', referencia_id=None)])
```

Salida.

```
[CitaEnTexto(cita_id='c1', texto_cita='(Edge et al., 2024)', tipo=<TipoCita.PARENTETICA:
'parentetica'>, pagina=1, fragmento_oracion='El grafo estructura el corpus.',
referencia_id=None), CitaEnTexto(cita_id='c2', texto_cita='(Edge et al., 2024)',
tipo=<TipoCita.PARENTETICA: 'parentetica'>, pagina=1, fragmento_oracion='La evaluación
se
hizo sobre preguntas globales.', referencia_id=None)]
```

Resultado. PASSED

test_citas_distintas_nunca_se_colapsan

Descripción. Dos citas distintas con el mismo fragmento de contexto no se colapsan.

Código.

```
def test_citas_distintas_nunca_se_colapsan(self, svc):
    dedup = svc._deduplicar_citas([
        self._cita("(Edge et al., 2024)", "Un fragmento.", "c1"),
        self._cita("(Fan et al., 2024)", "Un fragmento.", "c2"),
    ])
    assert len(dedup) == 2
```

Entrada.

```
EntidadExtractionService._deduplicar_citas([CitaEnTexto(cita_id='c1', texto_cita='(Edge et al., 2024)', tipo=<TipoCita.PARENTETICA: 'parentetica'>, pagina=1, fragmento_oracion='Un fragmento.', referencia_id=None), CitaEnTexto(cita_id='c2', texto_cita='(Fan et al., 2024)', tipo=<TipoCita.PARENTETICA: 'parentetica'>, pagina=1, fragmento_oracion='Un fragmento.', referencia_id=None)])
```

Salida.

```
[CitaEnTexto(cita_id='c1', texto_cita='(Edge et al., 2024)', tipo=<TipoCita.PARENTETICA: 'parentetica'>, pagina=1, fragmento_oracion='Un fragmento.', referencia_id=None), CitaEnTexto(cita_id='c2', texto_cita='(Fan et al., 2024)', tipo=<TipoCita.PARENTETICA: 'parentetica'>, pagina=1, fragmento_oracion='Un fragmento.', referencia_id=None)]
```

Resultado. PASSED

7.2.7. TestExtraerOracionFallback

Cuando el LLM no devuelve la aseveración, se recorta del propio texto.

test_recupera_la_oracion_que_rodea_a_la_cita

Descripción. Cuando el LLM no devuelve la aseveración, el respaldo recorta del propio bloque la oración que termina en la cita, con un máximo de 800 caracteres.

Código.

```
def test_recupera_la_oracion_que_rodea_a_la_cita(self, svc, texto_cuerpo):
    cita = "(Edge et al., 2024)"
    bloque = next(b for b in svc._dividir_en_bloques(texto_cuerpo) if cita in b)
    fragmento = svc._extraer_oracion_fallback(bloque, cita)
    assert cita in fragmento
    assert fragmento.endswith(cita)
    assert len(fragmento) <= 800
```

Entrada.

- 1) EntidadExtractionService._dividir_en_bloques(<cadena de 96 093 caracteres>
'UNIVERSIDAD
PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA
DE
COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE
COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta
'...)
- 2) EntidadExtractionService._extraer_oracion_fallback(<cadena de 6 953 caracteres>
'drásticamente el riesgo de alucinaciones y mejorando la confiabilidad\n\ngeneral del
sistema (Klesel y Wittmann 2025)\n\n3.3.2. GraphRAG\n\nGraphRAG es una técnica
avanzada que
integra la Generación\n\nAumentada por Recuperación (RAG) con datos estructurados
en\n\nform'..., '(Edge et al., 2024)')

Salida.

- 1) ['UNIVERSIDAD PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO
DE
INGENIERÍA DE COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL
DE\n\nINGENIERO DE COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con
evaluación experta en detección de\n\ninconsistencias en citaciones bibliográficas de
tesis,
UPAO 2024-2025\n\nLínea de Investigación:\n\nRobótica, automatización avanzada y

```

sistemas
  inteligentes.\n\nSub Línea de Investigación: Sistemas Inteligentes\n\nAutor: Alcalde
Lavado
  Matias Felipe\n\n**Jurado
evaluador:**\n\nPresidente:\n\nSecretario:\n\nVocal:\n\nApellidos,
  Nombres\n\nApellidos, Nombres\n\nAsesor:\n\nCueva Chavez, Walter\n\n[Código Orc ...
[99007
  caracteres en total]
2) 'En su implementación\n\nmás difundida, Microsoft GraphRAG (Edge et al., 2024)'

```

Resultado. PASSED

test_si_la_cita_no_esta_en_el_bloque_devuelve_vacio

Descripción. Si la cita no está en el bloque, el respaldo devuelve cadena vacía en lugar de un fragmento arbitrario.

Código.

```

def test_si_la_cita_no_esta_en_el_bloque_devuelve_vacio(self, svc, texto_cuerpo):
    bloque = svc._dividir_en_bloques(texto_cuerpo)[0]
    assert svc._extraer_oracion_fallback(bloque, "(Inexistente, 1999)") == ""

```

Entrada.

```

1) EntidadExtractionService._dividir_en_bloques(<cadena de 96 093 caracteres>
'UNIVERSIDAD
  PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA
DE
  COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE
  COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta
'...)
2) EntidadExtractionService._extraer_oracion_fallback(<cadena de 7 000 caracteres>
'UNIVERSIDAD
  PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO DE INGENIERÍA
DE
  COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL DE\n\nINGENIERO DE
  COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con evaluación experta
',
'...',
'(Inexistente, 1999)')

```

Salida.

```

1) ['UNIVERSIDAD PRIVADA ANTENOR ORREGO\n\nFACULTAD DE INGENIERÍA\n\nPROGRAMA DE ESTUDIO
DE
  INGENIERÍA DE COMPUTACIÓN Y SISTEMAS\n\nTESIS PARA OPTAR EL TÍTULO PROFESIONAL
DE\n\nINGENIERO DE COMPUTACIÓN Y SISTEMAS\n\nConcordancia de un sistema GraphRAG con
  evaluación experta en detección de\n\ninconsistencias en citaciones bibliográficas de
  tesis,
  UPAO 2024-2025\n\nLínea de Investigación:\n\nRobótica, automatización avanzada y
sistemas
  inteligentes.\n\nSub Línea de Investigación: Sistemas Inteligentes\n\nAutor: Alcalde
Lavado
  Matias Felipe\n\n**Jurado
evaluador:**\n\nPresidente:\n\nSecretario:\n\nVocal:\n\nApellidos,
  Nombres\n\nApellidos, Nombres\n\nAsesor:\n\nCueva Chavez, Walter\n\n[Código Orc ...
[99007
  caracteres en total]
2) ''

```

Resultado. PASSED

7.2.8. TestParsearJSON

Respuestas del LLM: JSONL, array y basura.

test_parsea_una_respuesta_jsonl

Descripción. Una respuesta del LLM en formato JSONL (un objeto por línea) se parsea en la lista de objetos correspondiente.

Código.

```
def test_parsea_una_respuesta_jsonl(self, svc):
    respuesta = (
        '{"autores": ["Ansari, S."], "anio": 2026, "titulo": "Compound Deception
in Elite Peer Review"}\n'
        '{"autores": ["Edge, D."], "anio": 2024, "titulo": "From Local to
Global"}\n'
    )
    objetos = svc._parsear_json(respuesta)
    assert len(objetos) == 2
    assert objetos[0]["anio"] == 2026
```

Entrada.

```
EntidadExtractionService._parsear_json('{"autores": ["Ansari, S."], "anio": 2026,
"titulo":
"Compound Deception in Elite Peer Review"}\n{"autores": ["Edge, D."], "anio": 2024,
"titulo": "From Local to Global"}')
```

Salida.

```
[{'autores': ['Ansari, S.'], 'anio': 2026, 'titulo': 'Compound Deception in Elite Peer
Review'}, {'autores': ['Edge, D.'], 'anio': 2024, 'titulo': 'From Local to Global'}]
```

Resultado. PASSED

test_parsea_un_array_envuelto_en_markdown

Descripción. Un array JSON envuelto en un bloque de código markdown se parsea correctamente.

Código.

```
def test_parsea_un_array_envuelto_en_markdown(self, svc):
    respuesta = ```json\n[{"texto_cita": "(Edge et al., 2024)", "tipo":
"PARENTETICA"}]\n```
    objetos = svc._parsear_json(respuesta)
    assert len(objetos) == 1
    assert objetos[0]["tipo"] == "PARENTETICA"
```

Entrada.

```
EntidadExtractionService._parsear_json(```json\n[{"texto_cita": "(Edge et al., 2024)",
"tipo":
"PARENTETICA"}]\n```)
```

Salida.

```
[{'texto_cita': '(Edge et al., 2024)', 'tipo': 'PARENTETICA'}]
```

Resultado. PASSED

test_una_respuesta_sin_json_no_rompe

Descripción. Una respuesta del LLM sin JSON devuelve una lista vacía en lugar de lanzar excepción.

Código.

```
def test_una_respuesta_sin_json_no_rompe(self, svc):
    assert svc._parsear_json("No he encontrado ninguna cita.") == []
```

Entrada.

```
EntidadExtractionService._parsear_json('No he encontrado ninguna cita.')
```

Salida.

```
[]
```

Resultado. PASSED

7.3. GrafoCargaService

Archivo de pruebas	Módulo bajo prueba	Pruebas
tests/unit/test_grafo_carga_service.py	app/services/grafos/grafos_carga_service.py	12

GrafoCargaService — vinculación automática cita → referencia.

Es la regla que decide qué cita pertenece a qué fuente, y de ella depende que la auditoría busque evidencia en el paper correcto. Neo4j está mockeado: no se abre ninguna conexión. Los apellidos y años son los de las referencias reales de la tesis.

7.3.1. TestNormalizadores

test_el_nombre_pierde_tildes_y_mayusculas

Descripción. La normalización de nombres elimina tildes y pasa a minúsculas.

Código.

```
def test_el_nombre_pierde_tildes_y_mayusculas(self):
    assert _normalizar_nombre("García, J.") == "garcia, j."
```

Entrada.

```
_normalizar_nombre('García, J.')
```

Salida.

```
'garcia, j.'
```

Resultado. PASSED

test_el_apellido_se_compara_sin_tildes

Descripción. Dos escrituras del mismo apellido, con y sin tilde, normalizan al mismo valor.

Código.

```
def test_el_apellido_se_compara_sin_tildes(self):
    assert _normalizar_apellido("Arévalo") == _normalizar_apellido("arevalo")
```

Entrada.

```
1) _normalizar_apellido('Arévalo')
2) _normalizar_apellido('arevalo')
```

Salida.

```
1) 'arevalo'
2) 'arevalo'
```

Resultado. PASSED

test_los_espacios_sobrantes_no_afectan

Descripción. Los espacios sobrantes alrededor de un apellido no alteran su forma normalizada.

Código.

```
def test_los_espacios_sobrantes_no_afectan(self):
    assert _normalizar_apellido(" Edge ") == "edge"
```

Entrada.

```
_normalizar_apellido(' Edge ')
```

Salida.

```
'edge'
```

Resultado. PASSED

7.3.2. TestVincularCitas**test_apellido_y_anio_exactos_vinculan_con_confianza_alta**

Descripción. Cuando el apellido y el año de la cita coinciden exactamente con los de una referencia, se crea el vínculo con confianza 0.95 y método apellido_anio_exacto.

Código.

```
def test_apellido_y_anio_exactos_vinculan_con_confianza_alta(self):
    resumen, sesion = _vincular(
        citas=[{"cita_id": "c1", "texto_cita": "(Edge et al., 2024)"}],
        referencias=[{"ref_id": "r1", "anio": 2024, "autores": ["Edge, D."]}],
    )
    assert resumen == {"vinculadas": 1, "no_vinculadas": 0, "total": 1}
    cita_id, ref_id, confianza, metodo = sesion.vinculos[0]
    assert (cita_id, ref_id) == ("c1", "r1")
    assert confianza == 0.95
    assert metodo == "apellido_anio_exacto"
```

Entrada.

```
GrafoCargaService.vincular_citas('doc-1')
```

Salida.

```
{'vinculadas': 1, 'no_vinculadas': 0, 'total': 1}
```

Resultado. PASSED

test_una_cita_de_dos_autores_vincula_por_el_primer

Descripción. Una cita de dos autores se vincula por el apellido del primer autor de la referencia, con confianza alta.

Código.

```
def test_una_cita_de_dos_autores_vincula_por_el_primer(self):
    resumen, sesion = _vincular(
        citas=[{"cita_id": "c1", "texto_cita": "(Klesel y Wittmann, 2025)"},
        referencias=[{"ref_id": "r1", "anio": 2025, "autores": ["Klesel, M.",
"Wittmann, H."]}],
    )
    assert resumen["vinculadas"] == 1
    assert sesion.vinculos[0][2] == 0.95
```

Entrada.

```
GrafoCargaService.vincular_citas('doc-1')
```

Salida.

```
{'vinculadas': 1, 'no_vinculadas': 0, 'total': 1}
```

Resultado. PASSED

test_un_apellido_truncado_vincula_con_confianza_menor

Descripción. Un apellido truncado por la extracción del PDF se vincula por prefijo con confianza 0.75 y método apellido_parcial_anio.

Código.

```
def test_un_apellido_truncado_vincula_con_confianza_menor(self):
    # El PDF puede partir el apellido; la estrategia 2 lo cubre por prefijo.
    resumen, sesion = _vincular(
        citas=[{"cita_id": "c1", "texto_cita": "(Wittman, 2025)"},
        referencias=[{"ref_id": "r1", "anio": 2025, "autores": ["Wittmann,
H."]}],
    )
    assert resumen["vinculadas"] == 1
    assert sesion.vinculos[0][2] == 0.75
    assert sesion.vinculos[0][3] == "apellido_parcial_anio"
```

Entrada.

```
GrafoCargaService.vincular_citas('doc-1')
```

Salida.

```
{'vinculadas': 1, 'no_vinculadas': 0, 'total': 1}
```

Resultado. PASSED

test_no_vincula_solo_por_coincidir_el_anio

Descripción. Coincidir únicamente en el año no basta para vincular: sin coincidencia de apellido no se crea el vínculo, evitando falsos positivos.

Código.

```
def test_no_vincula_solo_por_coincidir_el_anio(self):
    # Regresión: enlazar por año sin apellido producía falsos positivos.
```

```

resumen, sesion = _vincular(
    citas=[{"cita_id": "c1", "texto_cita": "(Zhao et al., 2024)"}],
    referencias=[{"ref_id": "r1", "anio": 2024, "autores": ["Edge, D."]}],
)
assert resumen == {"vinculadas": 0, "no_vinculadas": 1, "total": 1}
assert sesion.vinculos == []

```

Entrada.

```
GrafoCargaService.vincular_citas('doc-1')
```

Salida.

```
{'vinculadas': 0, 'no_vinculadas': 1, 'total': 1}
```

Resultado. PASSED

test_no_vincula_si_el_anio_no_coincide

Descripción. Si el año de la cita no coincide con el de la referencia, la cita queda sin vincular aunque el apellido coincida.

Código.

```

def test_no_vincula_si_el_anio_no_coincide(self):
    resumen, _ = _vincular(
        citas=[{"cita_id": "c1", "texto_cita": "(Edge et al., 2020)"}],
        referencias=[{"ref_id": "r1", "anio": 2024, "autores": ["Edge, D."]}],
    )
    assert resumen["no_vinculadas"] == 1

```

Entrada.

```
GrafoCargaService.vincular_citas('doc-1')
```

Salida.

```
{'vinculadas': 0, 'no_vinculadas': 1, 'total': 1}
```

Resultado. PASSED

test_una_cita_sin_anio_no_se_vincula

Descripción. Una cita sin año no se vincula a ninguna referencia.

Código.

```

def test_una_cita_sin_anio_no_se_vincula(self):
    resumen, _ = _vincular(
        citas=[{"cita_id": "c1", "texto_cita": "(Edge et al.)"}],
        referencias=[{"ref_id": "r1", "anio": 2024, "autores": ["Edge, D."]}],
    )
    assert resumen["no_vinculadas"] == 1

```

Entrada.

```
GrafoCargaService.vincular_citas('doc-1')
```

Salida.

```
{'vinculadas': 0, 'no_vinculadas': 1, 'total': 1}
```

Resultado. PASSED

test_sin_referencias_ninguna_cita_se_vincula

Descripción. Sin referencias en el documento, ninguna cita se vincula y todas se contabilizan como no vinculadas.

Código.

```
def test_sin_referencias_ninguna_cita_se_vincula(self):
    resumen, _ = _vincular(
        citas=[{"cita_id": "c1", "texto_cita": "(Edge et al., 2024)"},
        referencias=[],
    )
    assert resumen == {"vinculadas": 0, "no_vinculadas": 1, "total": 1}
```

Entrada.

```
GrafoCargaService.vincular_citas('doc-1')
```

Salida.

```
{'vinculadas': 0, 'no_vinculadas': 1, 'total': 1}
```

Resultado. PASSED

test_sin_citas_pendientes_no_hace_nada

Descripción. Sin citas pendientes no se ejecuta ninguna escritura en el grafo y el resumen queda en cero.

Código.

```
def test_sin_citas_pendientes_no_hace_nada(self):
    resumen, sesion = _vincular(citas=[], referencias=[{"ref_id": "r1", "anio":
2024, "autores": ["Edge, D."]}])
    assert resumen == {"vinculadas": 0, "no_vinculadas": 0, "total": 0}
    assert sesion.vinculos == []
```

Entrada.

```
GrafoCargaService.vincular_citas('doc-1')
```

Salida.

```
{'vinculadas': 0, 'no_vinculadas': 0, 'total': 0}
```

Resultado. PASSED

test_varias_citas_se_reparten_entre_sus_referencias

Descripción. Con varias citas y varias referencias, cada cita se vincula a la referencia que le corresponde y no a otra.

Código.

```
def test_varias_citas_se_reparten_entre_sus_referencias(self):
    resumen, sesion = _vincular(
        citas=[
            {"cita_id": "c1", "texto_cita": "(Edge et al., 2024)"},
            {"cita_id": "c2", "texto_cita": "(Fan et al., 2024)"},
            {"cita_id": "c3", "texto_cita": "(Ansari, 2026)"},
        ],
        referencias=[
            {"ref_id": "r1", "anio": 2024, "autores": ["Edge, D."]},
        ],
    )
```

```

        {"ref_id": "r2", "anio": 2024, "autores": ["Fan, W."]},
        {"ref_id": "r3", "anio": 2026, "autores": ["Ansari, S."]},
    ],
)
assert resumen["vinculadas"] == 3
assert {(c, r) for c, r, _, _ in sesion.vinculos} == {("c1", "r1"), ("c2",
"r2"), ("c3", "r3")}

```

Entrada.

```
GrafoCargaService.vincular_citas('doc-1')
```

Salida.

```
{'vinculadas': 3, 'no_vinculadas': 0, 'total': 3}
```

Resultado. PASSED

7.4. RecuperacionService

Archivo de pruebas	Módulo bajo prueba	Pruebas
tests/unit/test_recuperacion_service.py	app/services/recuperacion/recuperacion_service.py	8

RecuperacionService — recuperación híbrida (grafo + pgvector).

Lo que se prueba aquí es la decisión de qué evidencia se entrega al juez y con qué método. Neo4j y pgvector están mockeados; no se abre ninguna conexión ni se calcula ningún embedding real.

La regla crítica: la búsqueda vectorial se restringe al DOI de la referencia que la cita dice estar citando. Sin esa restricción, el sistema podría "encontrar evidencia" en un paper distinto.

7.4.1. TestConsultarCita

test_con_doi_y_chunks_devuelve_evidencia_por_metodo_hibrido

Descripción. Con referencia con DOI y fragmentos indexados, la recuperación devuelve evidencia por el método híbrido, con su similitud, el DOI y el identificador de la referencia.

Código.

```

def test_con_doi_y_chunks_devuelve_evidencia_por_metodo_hibrido(self):
    with (
        patch(_P_RUTA, return_value=_RUTA),
        patch(_P_NODOS, return_value=[]),
        patch(_P_FRAG, return_value=[_FRAGMENTO]),
        patch(_P_COSER, return_value=_FRAGMENTO["contenido"]),
    ):
        r = RecuperacionService().consultar_cita("doc-1", "c1")

        assert r.metodo == "hibrido"
        assert r.fragmento_relevante == _FRAGMENTO["contenido"]
        assert r.similitud == 0.87
        assert r.doi_referencia == DOI
        assert r.referencia_id == "r1"

```

Entrada.

```
RecuperacionService.consultar_cita('doc-1', 'c1')
```

Salida.

```
ResultadoRecuperacion(cita_id='c1', texto_cita='(Edge et al., 2024)',
    fragmento_relevante='GraphRAG outperforms naive RAG on global sensemaking questions.',
    similitud=0.87, nodos_grafo=[], doi_referencia='10.3390/educsci11040173',
    referencia_id='r1', metodo='hibrido', pagina_paper=4, chunk_index=12)
```

Resultado. PASSED

test_propaga_la_pagina_del_paper_de_la_que_sale_la_evidencia

Descripción. El resultado propaga la página del artículo fuente y el índice del fragmento del que procede la evidencia.

Código.

```
def test_propaga_la_pagina_del_paper_de_la_que_sale_la_evidencia(self):
    with (
        patch(_P_RUTA, return_value=_RUTA),
        patch(_P_NODOS, return_value=[]),
        patch(_P_FRAG, return_value=[_FRAGMENTO]),
        patch(_P_COSER, return_value=_FRAGMENTO["contenido"]),
    ):
        r = RecuperacionService().consultar_cita("doc-1", "c1")

        assert r.pagina_paper == 4
        assert r.chunk_index == 12
```

Entrada.

```
RecuperacionService.consultar_cita('doc-1', 'c1')
```

Salida.

```
ResultadoRecuperacion(cita_id='c1', texto_cita='(Edge et al., 2024)',
    fragmento_relevante='GraphRAG outperforms naive RAG on global sensemaking questions.',
    similitud=0.87, nodos_grafo=[], doi_referencia='10.3390/educsci11040173',
    referencia_id='r1', metodo='hibrido', pagina_paper=4, chunk_index=12)
```

Resultado. PASSED

test_la_busqueda_se_restringe_al_doi_de_la_referencia_citada

Descripción. La búsqueda vectorial se restringe a la clave derivada del DOI de la referencia citada, de modo que la evidencia no puede provenir de otro artículo.

Código.

```
def test_la_busqueda_se_restringe_al_doi_de_la_referencia_citada(self):
    with (
        patch(_P_RUTA, return_value=_RUTA),
        patch(_P_NODOS, return_value=[]),
        patch(_P_COSER, return_value=""),
        patch(_P_FRAG, return_value=[_FRAGMENTO]) as buscar,
    ):
        RecuperacionService().consultar_cita("doc-1", "c1")

        _query, doi_usado, _n = buscar.call_args[0]
        assert doi_usado == DOI_CHUNKS
```

Entrada.

```
RecuperacionService.consultar_cita('doc-1', 'c1')
```

Salida.

```
ResultadoRecuperacion(cita_id='c1', texto_cita='(Edge et al., 2024)',
    fragmento_relevante='',
    similitud=0.87, nodos_grafo=[], doi_referencia='10.3390/educsci11040173',
    referencia_id='r1', metodo='hibrido', pagina_paper=4, chunk_index=12)
```

Resultado. PASSED

test_busca_por_la_aseveracion_del_tesista_cuando_existe

Descripción. Si se dispone de la aseveración del tesista, la consulta vectorial se hace con ella y no con el texto de la cita.

Código.

```
def test_busca_por_la_aseveracion_del_tesista_cuando_existe(self):
    aseveracion = "GraphRAG mejora la recuperación en preguntas globales."
    with (
        patch(_P_RUTA, return_value=_RUTA),
        patch(_P_NODOS, return_value=[]),
        patch(_P_COSER, return_value=""),
        patch(_P_FRAG, return_value=[_FRAGMENTO]) as buscar,
    ):
        RecuperacionService().consultar_cita("doc-1", "c1",
            fragmento_oracion=aseveracion)

        query = buscar.call_args[0][0]
        assert query == aseveracion
```

Entrada.

```
RecuperacionService.consultar_cita('doc-1', 'c1', fragmento_oracion='GraphRAG mejora la
    recuperación en preguntas globales.')
```

Salida.

```
ResultadoRecuperacion(cita_id='c1', texto_cita='(Edge et al., 2024)',
    fragmento_relevante='',
    similitud=0.87, nodos_grafo=[], doi_referencia='10.3390/educsci11040173',
    referencia_id='r1', metodo='hibrido', pagina_paper=4, chunk_index=12)
```

Resultado. PASSED

test_sin_aseveracion_busca_por_el_texto_de_la_cita

Descripción. Sin aseveración utilizable, la consulta vectorial recae en el texto de la cita.

Código.

```
def test_sin_aseveracion_busca_por_el_texto_de_la_cita(self):
    with (
        patch(_P_RUTA, return_value=_RUTA),
        patch(_P_NODOS, return_value=[]),
        patch(_P_COSER, return_value=""),
        patch(_P_FRAG, return_value=[_FRAGMENTO]) as buscar,
    ):
        RecuperacionService().consultar_cita("doc-1", "c1", fragmento_oracion="")

    assert buscar.call_args[0][0] == "(Edge et al., 2024)"
```

Entrada.

```
RecuperacionService.consultar_cita('doc-1', 'c1', fragmento_oracion='')
```

Salida.

```
ResultadoRecuperacion(cita_id='c1', texto_cita='(Edge et al., 2024)',
    fragmento_relevante='',
    similitud=0.87, nodos_grafo=[], doi_referencia='10.3390/educsci11040173',
    referencia_id='r1', metodo='hibrido', pagina_paper=4, chunk_index=12)
```

Resultado. PASSED

test_sin_chunks_del_paper_cae_a_solo_grafo_y_sin_evidencia

Descripción. Si el artículo no tiene fragmentos indexados, el método pasa a solo_grafo, sin evidencia y con similitud cero, pero se conserva la referencia hallada.

Código.

```
def test_sin_chunks_del_paper_cae_a_solo_grafo_y_sin_evidencia(self):
    with (
        patch(_P_RUTA, return_value=_RUTA),
        patch(_P_NODOS, return_value=[]),
        patch(_P_FRAG, return_value=[]),
    ):
        r = RecuperacionService().consultar_cita("doc-1", "c1")

        assert r.metodo == "solo_grafo"
        assert r.fragmento_relevante == ""
        assert r.similitud == 0.0
        assert r.referencia_id == "r1"
```

Entrada.

```
RecuperacionService.consultar_cita('doc-1', 'c1')
```

Salida.

```
ResultadoRecuperacion(cita_id='c1', texto_cita='(Edge et al., 2024)',
    fragmento_relevante='',
    similitud=0.0, nodos_grafo=[], doi_referencia='10.3390/educsci11040173',
    referencia_id='r1',
    metodo='solo_grafo', pagina_paper=None, chunk_index=None)
```

Resultado. PASSED

test_una_cita_sin_referencia_vinculada_no_recupera_nada

Descripción. Una cita sin referencia vinculada devuelve método no_encontrado, sin referencia ni evidencia.

Código.

```
def test_una_cita_sin_referencia_vinculada_no_recupera_nada(self):
    with patch(_P_RUTA, return_value=None):
        r = RecuperacionService().consultar_cita("doc-1", "c1")

        assert r.metodo == "no_encontrado"
        assert r.referencia_id is None
        assert r.fragmento_relevante == ""
```

Entrada.

```
RecuperacionService.consultar_cita('doc-1', 'c1')
```

Salida.

```
ResultadoRecuperacion(cita_id='c1', texto_cita='', fragmento_relevante='', similitud=0.0,
    nodos_grafo=[], doi_referencia=None, referencia_id=None, metodo='no_encontrado',
    pagina_paper=None, chunk_index=None)
```

Resultado. PASSED

test_una_referencia_sin_doi_no_dispara_busqueda_vectorial

Descripción. Una referencia sin DOI no dispara ninguna búsqueda vectorial y el método queda en solo_grafo.

Código.

```
def test_una_referencia_sin_doi_no_dispara_busqueda_vectorial(self):
    ruta_sin_doi = {**_RUTA, "doi": None, "doi_raw": None}
    with (
        patch(_P_RUTA, return_value=ruta_sin_doi),
        patch(_P_NODOS, return_value=[]),
        patch(_P_FRAG) as buscar,
    ):
        r = RecuperacionService().consultar_cita("doc-1", "c1")

        buscar.assert_not_called()
        assert r.metodo == "solo_grafo"
```

Entrada.

```
RecuperacionService.consultar_cita('doc-1', 'c1')
```

Salida.

```
ResultadoRecuperacion(cita_id='c1', texto_cita='(Edge et al., 2024)',
    fragmento_relevante='',
    similitud=0.0, nodos_grafo=[], doi_referencia=None, referencia_id='r1',
    metodo='solo_grafo',
    pagina_paper=None, chunk_index=None)
```

Resultado. PASSED

7.5. AuditoriaService

Archivo de pruebas	Módulo bajo prueba	Pruebas
tests/unit/test_auditoria_service.py	app/services/auditoria/auditoria_service.py	14

AuditoriaService — emisión del veredicto por cita.

Ni el LLM ni Neo4j ni pgvector se tocan: todo está mockeado. Lo que se prueba es la política del auditor, que es donde está el riesgo real del sistema:

- sin referencia vinculada → NO_INFO (nunca se inventa una fuente) - paper no indexado → NO_INFO - referencia sin texto recuperado → NO_INFO (aunque la referencia exista) - con evidencia → se respeta el veredicto del juez

Un fallo aquí significaría dar por respaldada una cita sin haber leído la fuente.

7.5.1. TestCitaSinFuenteVerificable

Sin fuente no hay veredicto favorable posible: siempre NO_INFO.

test_sin_referencia_vinculada_devuelve_no_info

Descripción. Una cita sin referencia vinculada recibe veredicto NO_INFO, con método de recuperación no_encontrado y una justificación que alude a la ausencia de referencia.

Código.

```
def test_sin_referencia_vinculada_devuelve_no_info(self):
    with patch(_P_PERSISTIR):
        v = AuditoriaService()._auditar_cita(
            **{_KWARGS, "ref_id": None}, fragmento_oracion=ASEVERACION
        )
        assert v.veredicto == VeredictoTipo.NO_INFO
        assert v.metodo_recuperacion == "no_encontrado"
        assert "referencia" in v.justificacion.lower()
```

Entrada.

```
AuditoriaService._auditar_cita(documento_id='doc-1', cita_id='c1', texto_cita='(Edge et
al.,
    2024)', pagina=12, ref_id=None, ref_titulo='From Local to Global: A GraphRAG
Approach',
    ref_anio=2024, autores=['Edge, D.'], fragmento_oracion='GraphRAG supera a RAG clásico
en
preguntas globales (Edge et al., 2024).')
```

Salida.

```
VeredictoAuditoria(cita_id='c1', texto_cita='(Edge et al., 2024)', fragmento_oracion='',
    pagina=12, veredicto=<VeredictoTipo.NO_INFO: 'NO_INFO'>, justificacion='No se encontró
una
referencia bibliográfica vinculada a esta cita.', fragmento_evidencia='',
    fragmento_evidencia_es='', similitud=0.0, referencia_id=None, titulo_referencia=None,
    doi_referencia=None, autores_referencia=[], anio_referencia=None,
    metodo_recuperacion='no_encontrado', pagina_paper=None, faithfulness=None,
    answer_relevancy=None, context_precision=None)
```

Resultado. PASSED

test_no_llama_al_juez_si_no_hay_referencia

Descripción. Si la cita no tiene referencia vinculada, el juez LLM no llega a invocarse.

Código.

```
def test_no_llama_al_juez_si_no_hay_referencia(self):
    with patch(_P_PERSISTIR), patch(_P_LLM) as llm:
        AuditoriaService()._auditar_cita(**{_KWARGS, "ref_id": None})
        llm.assert_not_called()
```

Entrada.

```
AuditoriaService._auditar_cita(documento_id='doc-1', cita_id='c1', texto_cita='(Edge et
al.,
    2024)', pagina=12, ref_id=None, ref_titulo='From Local to Global: A GraphRAG
Approach',
    ref_anio=2024, autores=['Edge, D.'])
```

Salida.

```
VeredictoAuditoria(cita_id='c1', texto_cita='(Edge et al., 2024)', fragmento_oracion='',
    pagina=12, veredicto=<VeredictoTipo.NO_INFO: 'NO_INFO'>, justificacion='No se encontró
una
referencia bibliográfica vinculada a esta cita.', fragmento_evidencia='',
```

```

fragmento_evidencia_es='', similitud=0.0, referencia_id=None, titulo_referencia=None,
doi_referencia=None, autores_referencia=[], anio_referencia=None,
metodo_recuperacion='no_encontrado', pagina_paper=None, faithfulness=None,
answer_relevancy=None, context_precision=None)

```

Resultado. PASSED

test_paper_no_indexado_devuelve_no_info

Descripción. Si el artículo verificado no está indexado en el almacén vectorial, el veredicto es NO_INFO, la justificación lo atribuye a la base de conocimiento y no se consulta al juez.

Código.

```

def test_paper_no_indexado_devuelve_no_info(self):
    with (
        patch(_P_PERSISTIR),
        patch(_P_INDEXADO, return_value=False),
        patch(_P_LLM) as llm,
    ):
        v = AuditoriaService()._auditar_cita(**_KWARGS, doi_verificado=DOI)
        assert v.veredicto == VeredictoTipo.NO_INFO
        assert "base de conocimiento" in v.justificacion
        llm.assert_not_called()

```

Entrada.

```

AuditoriaService()._auditar_cita(documento_id='doc-1', cita_id='c1', texto_cita='(Edge et
al.,
    2024)', pagina=12, ref_id='r1', ref_titulo='From Local to Global: A GraphRAG
Approach',
    ref_anio=2024, autores=['Edge, D.'], doi_verificado='10.3390/educsci11040173')

```

Salida.

```

VeredictoAuditoria(cita_id='c1', texto_cita='(Edge et al., 2024)', fragmento_oracion='',
    pagina=12, veredicto=<VeredictoTipo.NO_INFO: 'NO_INFO'>, justificacion='El paper
citado no
    está disponible en la base de conocimiento.', fragmento_evidencia='',
    fragmento_evidencia_es='', similitud=0.0, referencia_id=None, titulo_referencia=None,
    doi_referencia=None, autores_referencia=[], anio_referencia=None,
    metodo_recuperacion='no_encontrado', pagina_paper=None, faithfulness=None,
    answer_relevancy=None, context_precision=None)

```

Resultado. PASSED

test_referencia_sin_texto_recuperado_devuelve_no_info

Descripción. Cuando la referencia existe pero no se recupera texto alguno, el veredicto es NO_INFO, se conserva el identificador de la referencia y no se consulta al juez.

Código.

```

def test_referencia_sin_texto_recuperado_devuelve_no_info(self):
    with (
        patch(_P_PERSISTIR),
        patch(_P_CONSULTAR, return_value=_resultado(metodo="solo_grafo",
fragmento="", similitud=0.0)),
        patch(_P_LLM) as llm,
    ):
        v = AuditoriaService()._auditar_cita(**_KWARGS)
        assert v.veredicto == VeredictoTipo.NO_INFO
        assert v.metodo_recuperacion == "solo_grafo"

```

```
assert v.referencia_id == "r1" # la referencia existe; el texto no
llm.assert_not_called()
```

Entrada.

```
AuditoriaService._auditar_cita(documento_id='doc-1', cita_id='c1', texto_cita='(Edge et
al.,
2024)', pagina=12, ref_id='r1', ref_titulo='From Local to Global: A GraphRAG
Approach',
ref_anio=2024, autores=['Edge, D.'])
```

Salida.

```
VeredictoAuditoria(cita_id='c1', texto_cita='(Edge et al., 2024)', fragmento_oracion='',
pagina=12, veredicto=<VeredictoTipo.NO_INFO: 'NO_INFO'>, justificacion='No se encontró
texto
del paper citado en la base de conocimiento. La referencia existe en el grafo pero no
hay
fragmento disponible para verificar.', fragmento_evidencia='',
fragmento_evidencia_es='',
similitud=0.0, referencia_id='r1', titulo_referencia='From Local to Global: A GraphRAG
Approach', doi_referencia='10.3390/educsci11040173', autores_referencia=['Edge, D.'],
anio_referencia=2024, metodo_recuperacion='solo_grafo', pagina_paper=4,
faithfulness=None,
answer_relevancy=None, context_precision=None)
```

Resultado. PASSED

7.5.2. TestCitaConEvidencia

Con evidencia recuperada, el veredicto es el del juez y se conserva la trazabilidad.

test_respetar_el_veredicto_de_respaldo

Descripción. Con evidencia recuperada, el veredicto SUPPORTS del juez se propaga sin alteración, junto con su justificación literal.

Código.

```
def test_respetar_el_veredicto_de_respaldo(self):
    with (
        patch(_P_PERSISTIR),
        patch(_P_INDEXADO, return_value=True),
        patch(_P_CONSULTAR, return_value=_resultado()),
        patch(_P_TRADUCIR, return_value="GraphRAG supera a RAG clásico."),
        patch(_P_LLM, return_value=(VeredictoTipo.SUPPORTS, "La fuente sostiene
la afirmación.")),
    ):
        v = AuditoriaService()._auditar_cita(
            **_KWARGS, doi_verificado=DOI, fragmento_oracion=ASEVERACION
        )
        assert v.veredicto == VeredictoTipo.SUPPORTS
        assert v.justificacion == "La fuente sostiene la afirmación."
```

Entrada.

```
AuditoriaService._auditar_cita(documento_id='doc-1', cita_id='c1', texto_cita='(Edge et
al.,
2024)', pagina=12, ref_id='r1', ref_titulo='From Local to Global: A GraphRAG
Approach',
ref_anio=2024, autores=['Edge, D.'], doi_verificado='10.3390/educsci11040173',
fragmento_oracion='GraphRAG supera a RAG clásico en preguntas globales (Edge et al.,
2024).')
```

Salida.

```
VeredictoAuditoria(cita_id='c1', texto_cita='(Edge et al., 2024)',
fragmento_oracion='GraphRAG
supera a RAG clásico en preguntas globales (Edge et al., 2024).', pagina=12,
veredicto=<VeredictoTipo.SUPPORTS: 'SUPPORTS'>, justificacion='La fuente sostiene la
afirmación.', fragmento_evidencia='GraphRAG outperforms naive RAG on global
sensemaking
questions.', fragmento_evidencia_es='GraphRAG supera a RAG clásico.', similitud=0.87,
referencia_id='r1', titulo_referencia='From Local to Global: A GraphRAG Approach',
doi_referencia='10.3390/educsci11040173', autores_referencia=['Edge, D.'],
anio_referencia=2024, metodo_recuperacion='hibrido', pagina_paper=4,
faithfulness=None,
answer_relevancy=None, ... [723 caracteres en total]
```

Resultado. PASSED

test_respeto_el_veredicto_de_refutacion

Descripción. Con evidencia recuperada, el veredicto REFUTES del juez se propaga sin alteración.

Código.

```
def test_respeto_el_veredicto_de_refutacion(self):
    with (
        patch(_P_PERSISTIR),
        patch(_P_CONSULTAR, return_value=_resultado()),
        patch(_P_TRADUCIR, return_value=""),
        patch(_P_LLM, return_value=(VeredictoTipo.REFUTES, "La fuente afirma lo
contrario.")),
    ):
        v = AuditoriaService()._auditar_cita(**_KWARGS,
fragmento_oracion=ASEVERACION)
        assert v.veredicto == VeredictoTipo.REFUTES
```

Entrada.

```
AuditoriaService._auditar_cita(documento_id='doc-1', cita_id='c1', texto_cita='(Edge et
al.,
2024)', pagina=12, ref_id='r1', ref_titulo='From Local to Global: A GraphRAG
Approach',
ref_anio=2024, autores=['Edge, D.'], fragmento_oracion='GraphRAG supera a RAG clásico
en
preguntas globales (Edge et al., 2024).')
```

Salida.

```
VeredictoAuditoria(cita_id='c1', texto_cita='(Edge et al., 2024)',
fragmento_oracion='GraphRAG
supera a RAG clásico en preguntas globales (Edge et al., 2024).', pagina=12,
veredicto=<VeredictoTipo.REFUTES: 'REFUTES'>, justificacion='La fuente afirma lo
contrario.', fragmento_evidencia='GraphRAG outperforms naive RAG on global sensemaking
questions.', fragmento_evidencia_es='', similitud=0.87, referencia_id='r1',
titulo_referencia='From Local to Global: A GraphRAG Approach',
doi_referencia='10.3390/educsci11040173', autores_referencia=['Edge, D.'],
anio_referencia=2024, metodo_recuperacion='hibrido', pagina_paper=4,
faithfulness=None,
answer_relevancy=None, context_precision=None)
```

Resultado. PASSED

test_adjunta_la_evidencia_y_la_pagina_del_paper

Descripción. El veredicto emitido adjunta el fragmento de evidencia, la página del artículo, la similitud y el método de recuperación empleado.

Código.

```
def test_adjunta_la_evidencia_y_la_pagina_del_paper(self):
    with (
        patch(_P_PERSISTIR),
        patch(_P_CONSULTAR, return_value=_resultado()),
        patch(_P_TRADUCIR, return_value=""),
        patch(_P_LLM, return_value=(VeredictoTipo.SUPPORTS, "Respaldada.")),
    ):
        v = AuditoriaService()._auditar_cita(**_KWARGS,
        fragmento_oracion=ASEVERACION)
        assert v.fragmento_evidencia == EVIDENCIA
        assert v.pagina_paper == 4
        assert v.similitud == 0.87
        assert v.metodo_recuperacion == "hibrido"
```

Entrada.

```
AuditoriaService()._auditar_cita(documento_id='doc-1', cita_id='c1', texto_cita='(Edge et
al.,
    2024)', pagina=12, ref_id='r1', ref_titulo='From Local to Global: A GraphRAG
Approach',
    ref_anio=2024, autores=['Edge, D.'], fragmento_oracion='GraphRAG supera a RAG clásico
en
    preguntas globales (Edge et al., 2024).')
```

Salida.

```
VeredictoAuditoria(cita_id='c1', texto_cita='(Edge et al., 2024)',
fragmento_oracion='GraphRAG
    supera a RAG clásico en preguntas globales (Edge et al., 2024).', pagina=12,
veredicto=<VeredictoTipo.SUPPORTS: 'SUPPORTS'>, justificacion='Respaldada.',
    fragmento_evidencia='GraphRAG outperforms naive RAG on global sensemaking questions.',
    fragmento_evidencia_es='', similitud=0.87, referencia_id='r1', titulo_referencia='From
Local
    to Global: A GraphRAG Approach', doi_referencia='10.3390/educsci11040173',
    autores_referencia=['Edge, D.'], anio_referencia=2024, metodo_recuperacion='hibrido',
    pagina_paper=4, faithfulness=None, answer_relevancy=None, context_precision=None)
```

Resultado. PASSED

test_conserva_la_aseveracion_del_tesista_en_el_veredicto

Descripción. El veredicto conserva la aseveración del tesista sobre la que se pronuncia.

Código.

```
def test_conserva_la_aseveracion_del_tesista_en_el_veredicto(self):
    with (
        patch(_P_PERSISTIR),
        patch(_P_CONSULTAR, return_value=_resultado()),
        patch(_P_TRADUCIR, return_value=""),
        patch(_P_LLM, return_value=(VeredictoTipo.SUPPORTS, "Respaldada.")),
    ):
        v = AuditoriaService()._auditar_cita(**_KWARGS,
        fragmento_oracion=ASEVERACION)
        assert v.fragmento_oracion == ASEVERACION
```

Entrada.

```
AuditoriaService._auditar_cita(documento_id='doc-1', cita_id='c1', texto_cita='(Edge et al., 2024)', pagina=12, ref_id='r1', ref_titulo='From Local to Global: A GraphRAG Approach', ref_anio=2024, autores=['Edge, D.'], fragmento_oracion='GraphRAG supera a RAG clásico en preguntas globales (Edge et al., 2024).')
```

Salida.

```
VeredictoAuditoria(cita_id='c1', texto_cita='(Edge et al., 2024)', fragmento_oracion='GraphRAG supera a RAG clásico en preguntas globales (Edge et al., 2024).', pagina=12, veredicto=<VeredictoTipo.SUPPORTS: 'SUPPORTS'>, justificacion='Respaldada.', fragmento_evidencia='GraphRAG outperforms naive RAG on global sensemaking questions.', fragmento_evidencia_es='', similitud=0.87, referencia_id='r1', titulo_referencia='From Local to Global: A GraphRAG Approach', doi_referencia='10.3390/educsci11040173', autores_referencia=['Edge, D.'], anio_referencia=2024, metodo_recuperacion='hibrido', pagina_paper=4, faithfulness=None, answer_relevancy=None, context_precision=None)
```

Resultado. PASSED

test_arrastra_los_datos_de_la_fuente_usada

Descripción. El veredicto arrastra el DOI, el título, el año y los autores de la referencia empleada, garantizando la trazabilidad de la fuente.

Código.

```
def test_arrastra_los_datos_de_la_fuente_usada(self):
    with (
        patch(_P_PERSISTIR),
        patch(_P_CONSULTAR, return_value=_resultado()),
        patch(_P_TRADUCIR, return_value=""),
        patch(_P_LLM, return_value=(VeredictoTipo.SUPPORTS, "Respaldada.")),
    ):
        v = AuditoriaService()._auditar_cita(**_KWARGS,
        fragmento_oracion=ASEVERACION)
        assert v.doi_referencia == DOI
        assert v.titulo_referencia == "From Local to Global: A GraphRAG Approach"
        assert v.anio_referencia == 2024
        assert v.autores_referencia == ["Edge, D."]
```

Entrada.

```
AuditoriaService._auditar_cita(documento_id='doc-1', cita_id='c1', texto_cita='(Edge et al., 2024)', pagina=12, ref_id='r1', ref_titulo='From Local to Global: A GraphRAG Approach', ref_anio=2024, autores=['Edge, D.'], fragmento_oracion='GraphRAG supera a RAG clásico en preguntas globales (Edge et al., 2024).')
```

Salida.

```
VeredictoAuditoria(cita_id='c1', texto_cita='(Edge et al., 2024)', fragmento_oracion='GraphRAG supera a RAG clásico en preguntas globales (Edge et al., 2024).', pagina=12, veredicto=<VeredictoTipo.SUPPORTS: 'SUPPORTS'>, justificacion='Respaldada.', fragmento_evidencia='GraphRAG outperforms naive RAG on global sensemaking questions.', fragmento_evidencia_es='', similitud=0.87, referencia_id='r1', titulo_referencia='From
```

```

Local
to Global: A GraphRAG Approach', doi_referencia='10.3390/educsci11040173',
autores_referencia=['Edge, D.'], anio_referencia=2024, metodo_recuperacion='hibrido',
pagina_paper=4, faithfulness=None, answer_relevancy=None, context_precision=None)

```

Resultado. PASSED

test_el_juez_recibe_la_aseveracion_y_la_evidencia

Descripción. El juez LLM recibe tanto la aseveración del tesista como el fragmento de evidencia recuperado.

Código.

```

def test_el_juez_recibe_la_aseveracion_y_la_evidencia(self):
    with (
        patch(_P_PERSISTIR),
        patch(_P_CONSULTAR, return_value=_resultado()),
        patch(_P_TRADUCIR, return_value=""),
        patch(_P_LLM, return_value=(VeredictoTipo.SUPPORTS, "Respaldada.")) as
    llm,
    ):
        AuditoriaService()._auditar_cita(**_KWARGS,
        fragmento_oracion=ASEVERACION)
        kwargs = llm.call_args.kwargs
        assert kwargs["fragmento_oracion"] == ASEVERACION
        assert kwargs["fragmento"] == EVIDENCIA

```

Entrada.

```

AuditoriaService()._auditar_cita(documento_id='doc-1', cita_id='c1', texto_cita='(Edge et
al.,
2024)', pagina=12, ref_id='r1', ref_titulo='From Local to Global: A GraphRAG
Approach',
ref_anio=2024, autores=['Edge, D.'], fragmento_oracion='GraphRAG supera a RAG clásico
en
preguntas globales (Edge et al., 2024).')

```

Salida.

```

VeredictoAuditoria(cita_id='c1', texto_cita='(Edge et al., 2024)',
fragmento_oracion='GraphRAG
supera a RAG clásico en preguntas globales (Edge et al., 2024).', pagina=12,
veredicto=<VeredictoTipo.SUPPORTS: 'SUPPORTS'>, justificacion='Respaldada.',
fragmento_evidencia='GraphRAG outperforms naive RAG on global sensemaking questions.',
fragmento_evidencia_es='', similitud=0.87, referencia_id='r1', titulo_referencia='From
Local
to Global: A GraphRAG Approach', doi_referencia='10.3390/educsci11040173',
autores_referencia=['Edge, D.'], anio_referencia=2024, metodo_recuperacion='hibrido',
pagina_paper=4, faithfulness=None, answer_relevancy=None, context_precision=None)

```

Resultado. PASSED

7.5.3. TestParsearRespuestaDelJuez

El parseo de la respuesta del LLM nunca debe romper la auditoría.

test_parsea_un_respaldo

Descripción. Una respuesta del juez con veredicto SUPPORTS se parsea en el tipo de veredicto y la justificación correspondientes.

Código.

```
def test_parsea_un_respaldo(self):
    v, j = AuditoriaService._parsear_respuesta_llm(
        "VERDICT: SUPPORTS\nJUSTIFICATION: El fragmento contiene la cifra
citada."
    )
    assert v == VeredictoTipo.SUPPORTS
    assert j == "El fragmento contiene la cifra citada."
```

Entrada.

```
AuditoriaService._parsear_respuesta_llm('VERDICT: SUPPORTS\nJUSTIFICATION: El fragmento
contiene la cifra citada.')
```

Salida.

```
(<VeredictoTipo.SUPPORTS: 'SUPPORTS'>, 'El fragmento contiene la cifra citada.')
```

Resultado. PASSED

test_parsea_una_refutacion

Descripción. Una respuesta del juez con veredicto REFUTES se parsea al tipo REFUTES.

Código.

```
def test_parsea_una_refutacion(self):
    v, _ = AuditoriaService._parsear_respuesta_llm(
        "VERDICT: REFUTES\nJUSTIFICATION: La fuente afirma lo contrario."
    )
    assert v == VeredictoTipo.REFUTES
```

Entrada.

```
AuditoriaService._parsear_respuesta_llm('VERDICT: REFUTES\nJUSTIFICATION: La fuente
afirma lo
contrario.')
```

Salida.

```
(<VeredictoTipo.REFUTES: 'REFUTES'>, 'La fuente afirma lo contrario.')
```

Resultado. PASSED

test_parsea_una_falta_de_evidencia

Descripción. Una respuesta del juez con veredicto NO_INFO se parsea al tipo NO_INFO.

Código.

```
def test_parsea_una_falta_de_evidencia(self):
    v, _ = AuditoriaService._parsear_respuesta_llm(
        "VERDICT: NO_INFO\nJUSTIFICATION: El fragmento no trata la afirmación."
    )
    assert v == VeredictoTipo.NO_INFO
```

Entrada.

```
AuditoriaService._parsear_respuesta_llm('VERDICT: NO_INFO\nJUSTIFICATION: El fragmento no
trata
la afirmación.')
```

Salida.

```
(<VeredictoTipo.NO_INFO: 'NO_INFO'>, 'El fragmento no trata la afirmación.')
```


Resultado. PASSED

test_una_respuesta_inesperada_cae_en_no_info

Descripción. Una respuesta del juez que no sigue el formato esperado se resuelve como NO_INFO en lugar de romper la auditoría.

Código.

```
def test_una_respuesta_inesperada_cae_en_no_info(self):
    v, _ = AuditoriaService._parsear_respuesta_llm("No estoy seguro de esta cita.")
    assert v == VeredictoTipo.NO_INFO
```

Entrada.

```
AuditoriaService._parsear_respuesta_llm('No estoy seguro de esta cita.')
```

Salida.

```
(<VeredictoTipo.NO_INFO: 'NO_INFO'>, "Could not interpret the verifier's response.")
```

Resultado. PASSED

7.6. metricas_service

Archivo de pruebas	Módulo bajo prueba	Pruebas
tests/unit/test_metricas_service.py	app/services/evaluacion/metricas_service.py	12

metricas_service — Kappa de Cohen, F1 y matriz de confusión.

Es scikit-learn puro sobre pares (predicción del sistema, etiqueta del experto). Sin LLM y sin RAGAS. Aquí no interviene la tesis: lo que se prueba es la aritmética que sustenta la vista /admin/evaluacion.

7.6.1. TestAcuerdoPerfecto

test_kappa_uno_cuando_sistema_y_experto_coinciden_en_todo

Descripción. Con acuerdo total entre sistema y experto, el Kappa de Cohen es 1.0 y todos los casos se cuentan como aciertos.

Código.

```
def test_kappa_uno_cuando_sistema_y_experto_coinciden_en_todo(self):
    pares = [("SUPPORTS", "SUPPORTS"), ("REFUTES", "REFUTES"), ("NO_INFO", "NO_INFO")]
    r = calcular_metricas(pares)
    assert r["kappa_cohen"] == 1.0
    assert r["aciertos"] == 3
    assert r["total_evaluadas"] == 3
```

Entrada.

```
calcular_metricas([('SUPPORTS', 'SUPPORTS'), ('REFUTES', 'REFUTES'), ('NO_INFO', 'NO_INFO')])
```

Salida.

```
{'total_evaluadas': 3, 'aciertos': 3, 'kappa_cohen': 1.0, 'matriz': [[1, 0, 0], [0, 1, 0], [0, 0, 1]], 'labels': ['SUPPORTS', 'REFUTES', 'NO_INFO'], 'por_categoria': [{'categoria': 'SUPPORTS', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 1}, {'categoria': 'REFUTES', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 1}, {'categoria': 'NO_INFO', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 1}], 'macro': {'categoria': 'macro', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 3}, 'weighted': {'categoria': 'weighted', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 3}}
```

Resultado. PASSED

test_f1_uno_en_acuerdo_perfecto

Descripción. Con acuerdo total, el F1 macro y el F1 ponderado son 1.0.

Código.

```
def test_f1_uno_en_acuerdo_perfecto(self):
    pares = [("SUPPORTS", "SUPPORTS"), ("REFUTES", "REFUTES"), ("NO_INFO", "NO_INFO")]
    r = calcular_metricas(pares)
    assert r["macro"]["f1"] == 1.0
    assert r["weighted"]["f1"] == 1.0
```

Entrada.

```
calcular_metricas([('SUPPORTS', 'SUPPORTS'), ('REFUTES', 'REFUTES'), ('NO_INFO', 'NO_INFO')])
```

Salida.

```
{'total_evaluadas': 3, 'aciertos': 3, 'kappa_cohen': 1.0, 'matriz': [[1, 0, 0], [0, 1, 0], [0, 0, 1]], 'labels': ['SUPPORTS', 'REFUTES', 'NO_INFO'], 'por_categoria': [{'categoria': 'SUPPORTS', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 1}, {'categoria': 'REFUTES', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 1}, {'categoria': 'NO_INFO', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 1}], 'macro': {'categoria': 'macro', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 3}, 'weighted': {'categoria': 'weighted', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 3}}
```

Resultado. PASSED

7.6.2. TestDesacuerdo

test_kappa_cero_cuando_solo_hay_una_clase_presente

Descripción. Cuando solo hay una clase presente, scikit-learn devuelve NaN y el servicio lo normaliza a 0.0.

Código.

```
def test_kappa_cero_cuando_solo_hay_una_clase_presente(self):
    # cohen_kappa_score devuelve NaN; el servicio debe normalizarlo a 0.0
    pares = [("SUPPORTS", "SUPPORTS"), ("SUPPORTS", "SUPPORTS")]
    assert calcular_metricas(pares)["kappa_cohen"] == 0.0
```

Entrada.

```
calcular_metricas([('SUPPORTS', 'SUPPORTS'), ('SUPPORTS', 'SUPPORTS')])
```

Salida.

```
{'total_evaluadas': 2, 'aciertos': 2, 'kappa_cohen': 0.0, 'matriz': [[2, 0, 0], [0, 0, 0], [0, 0, 0]], 'labels': ['SUPPORTS', 'REFUTES', 'NO_INFO'], 'por_categoria': [{'categoria': 'SUPPORTS', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 2}, {'categoria': 'REFUTES', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}, {'categoria': 'NO_INFO', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}], 'macro': {'categoria': 'macro', 'precision': 0.333, 'recall': 0.333, 'f1': 0.333, 'soporte': 2}, 'weighted': {'categoria': 'weighted', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 2}}
```

Resultado. PASSED

test_kappa_negativo_cuando_el_sistema_falla_sistematicamente

Descripción. Cuando el sistema se equivoca sistemáticamente, el Kappa es negativo y no se contabiliza ningún acierto.

Código.

```
def test_kappa_negativo_cuando_el_sistema_falla_sistematicamente(self):
    pares = [("SUPPORTS", "REFUTES"), ("REFUTES", "SUPPORTS")] * 3
    r = calcular_metricas(pares)
    assert r["kappa_cohen"] < 0
    assert r["aciertos"] == 0
```

Entrada.

```
calcular_metricas([('SUPPORTS', 'REFUTES'), ('REFUTES', 'SUPPORTS'), ('SUPPORTS', 'REFUTES'), ('REFUTES', 'SUPPORTS'), ('SUPPORTS', 'REFUTES'), ('REFUTES', 'SUPPORTS')])
```

Salida.

```
{'total_evaluadas': 6, 'aciertos': 0, 'kappa_cohen': -1.0, 'matriz': [[0, 3, 0], [3, 0, 0], [0, 0, 0]], 'labels': ['SUPPORTS', 'REFUTES', 'NO_INFO'], 'por_categoria': [{'categoria': 'SUPPORTS', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 3}, {'categoria': 'REFUTES', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 3}, {'categoria': 'NO_INFO', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}], 'macro': {'categoria': 'macro', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 6}, 'weighted': {'categoria': 'weighted', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 6}}
```

Resultado. PASSED

test_cuenta_los_aciertos_reales

Descripción. Con dos aciertos y dos fallos, se contabilizan dos aciertos y el Kappa queda estrictamente entre 0 y 1.

Código.

```
def test_cuenta_los_aciertos_reales(self):
    pares = [
        ("SUPPORTS", "SUPPORTS"), # acierto
        ("SUPPORTS", "REFUTES"), # fallo
        ("NO_INFO", "NO_INFO"), # acierto
        ("REFUTES", "NO_INFO"), # fallo
```

```
]
r = calcular_metricas(pares)
assert r["aciertos"] == 2
assert 0 < r["kappa_cohen"] < 1
```

Entrada.

```
calcular_metricas([('SUPPORTS', 'SUPPORTS'), ('SUPPORTS', 'REFUTES'), ('NO_INFO',
'NO_INFO'),
('REFUTES', 'NO_INFO')])
```

Salida.

```
{'total_evaluadas': 4, 'aciertos': 2, 'kappa_cohen': 0.273, 'matriz': [[1, 0, 0], [1, 0,
0],
[0, 1, 1]], 'labels': ['SUPPORTS', 'REFUTES', 'NO_INFO'], 'por_categoria':
[{'categoria':
'SUPPORTS', 'precision': 0.5, 'recall': 1.0, 'f1': 0.667, 'soporte': 1}, {'categoria':
'REFUTES', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 1}, {'categoria':
'NO_INFO', 'precision': 1.0, 'recall': 0.5, 'f1': 0.667, 'soporte': 2}], 'macro':
{'categoria': 'macro', 'precision': 0.5, 'recall': 0.5, 'f1': 0.444, 'soporte': 4},
'weighted': {'categoria': 'weighted', 'precision': 0.625, 'recall': 0.5, 'f1': 0.5,
'soporte': 4}}
```

Resultado. PASSED

7.6.3. TestMatrizDeConfusion

test_la_matriz_es_de_tres_por_tres

Descripción. La matriz de confusión es de tres por tres y sus etiquetas son las tres categorías del sistema.

Código.

```
def test_la_matriz_es_de_tres_por_tres(self):
    r = calcular_metricas([("SUPPORTS", "SUPPORTS")])
    assert len(r["matriz"]) == 3
    assert all(len(fila) == 3 for fila in r["matriz"])
    assert r["labels"] == LABELS
```

Entrada.

```
calcular_metricas([('SUPPORTS', 'SUPPORTS')])
```

Salida.

```
{'total_evaluadas': 1, 'aciertos': 1, 'kappa_cohen': 0.0, 'matriz': [[1, 0, 0], [0, 0,
0], [0,
0, 0]], 'labels': ['SUPPORTS', 'REFUTES', 'NO_INFO'], 'por_categoria': [{'categoria':
'SUPPORTS', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 1}, {'categoria':
'REFUTES', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}, {'categoria':
'NO_INFO', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}], 'macro':
{'categoria': 'macro', 'precision': 0.333, 'recall': 0.333, 'f1': 0.333, 'soporte':
1},
'weighted': {'categoria': 'weighted', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0,
'soporte':
1}}
```

Resultado. PASSED

test_las_filas_son_el_experto_y_las_columnas_el_sistema

Descripción. En la matriz de confusión las filas corresponden a la etiqueta del experto y las columnas a la predicción del sistema.

Código.

```
def test_las_filas_son_el_experto_y_las_columnas_el_sistema(self):
    # El experto dijo REFUTES y el sistema dijo NO_INFO: fila REFUTES, columna
    NO_INFO.
    r = calcular_metricas([("NO_INFO", "REFUTES")])
    fila_refutes = LABELS.index("REFUTES")
    col_no_info = LABELS.index("NO_INFO")
    assert r["matriz"][fila_refutes][col_no_info] == 1
```

Entrada.

```
calcular_metricas([('NO_INFO', 'REFUTES')])
```

Salida.

```
{'total_evaluadas': 1, 'aciertos': 0, 'kappa_cohen': 0.0, 'matriz': [[0, 0, 0], [0, 0,
1], [0,
0, 0]], 'labels': ['SUPPORTS', 'REFUTES', 'NO_INFO'], 'por_categoria': [{'categoria':
'SUPPORTS', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}, {'categoria':
'REFUTES', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 1}, {'categoria':
'NO_INFO', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}], 'macro':
{'categoria': 'macro', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 1},
'weighted': {'categoria': 'weighted', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0,
'soporte':
1}}
```

Resultado. PASSED

test_la_suma_de_la_matriz_es_el_total_evaluado

Descripción. La suma de todas las celdas de la matriz coincide con el total de casos evaluados.

Código.

```
def test_la_suma_de_la_matriz_es_el_total_evaluado(self):
    pares = [("SUPPORTS", "SUPPORTS"), ("REFUTES", "NO_INFO"), ("NO_INFO",
"NO_INFO")]
    r = calcular_metricas(pares)
    assert sum(sum(fila) for fila in r["matriz"]) == r["total_evaluadas"] == 3
```

Entrada.

```
calcular_metricas([('SUPPORTS', 'SUPPORTS'), ('REFUTES', 'NO_INFO'), ('NO_INFO',
'NO_INFO')])
```

Salida.

```
{'total_evaluadas': 3, 'aciertos': 2, 'kappa_cohen': 0.5, 'matriz': [[1, 0, 0], [0, 0,
0], [0,
1, 1]], 'labels': ['SUPPORTS', 'REFUTES', 'NO_INFO'], 'por_categoria': [{'categoria':
'SUPPORTS', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 1}, {'categoria':
'REFUTES', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}, {'categoria':
'NO_INFO', 'precision': 1.0, 'recall': 0.5, 'f1': 0.667, 'soporte': 2}], 'macro':
{'categoria': 'macro', 'precision': 0.667, 'recall': 0.5, 'f1': 0.556, 'soporte': 3},
'weighted': {'categoria': 'weighted', 'precision': 1.0, 'recall': 0.667, 'f1': 0.778,
'soporte': 3}}
```

Resultado. PASSED

7.6.4. TestMetricasPorCategoria

test_devuelve_las_tres_categorias_en_orden_fijo

Descripción. Las métricas por categoría se devuelven siempre en el mismo orden de etiquetas.

Código.

```
def test_devuelve_las_tres_categorias_en_orden_fijo(self):
    r = calcular_metricas([("SUPPORTS", "SUPPORTS")])
    assert [c["categoria"] for c in r["por_categoria"]] == LABELS
```

Entrada.

```
calcular_metricas([('SUPPORTS', 'SUPPORTS')])
```

Salida.

```
{'total_evaluadas': 1, 'aciertos': 1, 'kappa_cohen': 0.0, 'matriz': [[1, 0, 0], [0, 0, 0], [0, 0, 0]], 'labels': ['SUPPORTS', 'REFUTES', 'NO_INFO'], 'por_categoria': [{'categoria': 'SUPPORTS', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 1}, {'categoria': 'REFUTES', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}, {'categoria': 'NO_INFO', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}], 'macro': {'categoria': 'macro', 'precision': 0.333, 'recall': 0.333, 'f1': 0.333, 'soporte': 1}, 'weighted': {'categoria': 'weighted', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 1}}
```

Resultado. PASSED

test_el_soporte_es_el_numero_de_casos_del_experto

Descripción. El soporte de cada categoría es el número de casos que el experto etiquetó con ella.

Código.

```
def test_el_soporte_es_el_numero_de_casos_del_experto(self):
    pares = [("SUPPORTS", "SUPPORTS"), ("REFUTES", "SUPPORTS"), ("NO_INFO", "NO_INFO")]
    r = calcular_metricas(pares)
    soporte = {c["categoria"]: c["soporte"] for c in r["por_categoria"]}
    assert soporte["SUPPORTS"] == 2 # el experto etiquetó dos como SUPPORTS
    assert soporte["NO_INFO"] == 1
```

Entrada.

```
calcular_metricas([('SUPPORTS', 'SUPPORTS'), ('REFUTES', 'SUPPORTS'), ('NO_INFO', 'NO_INFO')])
```

Salida.

```
{'total_evaluadas': 3, 'aciertos': 2, 'kappa_cohen': 0.5, 'matriz': [[1, 1, 0], [0, 0, 0], [0, 1, 1]], 'labels': ['SUPPORTS', 'REFUTES', 'NO_INFO'], 'por_categoria': [{'categoria': 'SUPPORTS', 'precision': 1.0, 'recall': 0.5, 'f1': 0.667, 'soporte': 2}, {'categoria': 'REFUTES', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}, {'categoria': 'NO_INFO', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 1}], 'macro': {'categoria': 'macro', 'precision': 0.667, 'recall': 0.5, 'f1': 0.556, 'soporte': 3}, 'weighted': {'categoria': 'weighted', 'precision': 1.0, 'recall': 0.667, 'f1': 0.778, 'soporte': 3}}
```

Resultado. PASSED

test_una_categoria_sin_casos_no_lanza_division_por_cero

Descripción. Una categoría sin ningún caso obtiene F1 0.0 y soporte 0, sin provocar una división por cero.

Código.

```
def test_una_categoria_sin_casos_no_lanza_division_por_cero(self):
    r = calcular_metricas([("SUPPORTS", "SUPPORTS")])
    refutes = next(c for c in r["por_categoria"] if c["categoria"] == "REFUTES")
    assert refutes["f1"] == 0.0
    assert refutes["soporte"] == 0
```

Entrada.

```
calcular_metricas([('SUPPORTS', 'SUPPORTS')])
```

Salida.

```
{'total_evaluadas': 1, 'aciertos': 1, 'kappa_cohen': 0.0, 'matriz': [[1, 0, 0], [0, 0,
0], [0,
0, 0]], 'labels': ['SUPPORTS', 'REFUTES', 'NO_INFO'], 'por_categoria': [{'categoria':
'SUPPORTS', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'soporte': 1}, {'categoria':
'REFUTES', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}, {'categoria':
'NO_INFO', 'precision': 0.0, 'recall': 0.0, 'f1': 0.0, 'soporte': 0}], 'macro':
{'categoria': 'macro', 'precision': 0.333, 'recall': 0.333, 'f1': 0.333, 'soporte':
1},
'weighted': {'categoria': 'weighted', 'precision': 1.0, 'recall': 1.0, 'f1': 1.0,
'soporte':
1}}
```

Resultado. PASSED

7.6.5. TestValidacion

test_sin_pares_lanza_error_explicito

Descripción. Sin pares de evaluación, el cálculo lanza un ValueError explícito en lugar de devolver métricas sin sentido.

Código.

```
def test_sin_pares_lanza_error_explicito(self):
    with pytest.raises(ValueError):
        calcular_metricas([])
```

Entrada.

```
calcular_metricas([])
```

Salida.

```
lanza ValueError | 'Se requiere al menos un par (prediccion, etiqueta) para evaluar.'
```

Resultado. PASSED

7.7. doi_utils

Archivo de pruebas	Módulo bajo prueba	Pruebas
tests/unit/test_doi_utils.py	app/services/externo/doi_utils.py	12

doi_utils — normalización de DOI.

Es la pieza que decide si una referencia de la tesis y un paper de CrossRef son la misma fuente, y bajo qué clave se guardan sus chunks en pgvector. Los DOIs usados aquí son DOIs reales tomados de la bibliografía de la tesis.

7.7.1. TestNormalizarDOI

test_un_doi_limpio_no_cambia

Descripción. Un DOI ya normalizado no se altera.

Código.

```
def test_un_doi_limpio_no_cambia(self):
    assert normalizar_doi(DOI_TESIS) == DOI_TESIS
```

Entrada.

```
normalizar_doi('10.3390/educsci11040173')
```

Salida.

```
'10.3390/educsci11040173'
```

Resultado. PASSED

test_quita_el_prefijo_https_doi_org

Descripción. Se elimina el prefijo https://doi.org/ del DOI.

Código.

```
def test_quita_el_prefijo_https_doi_org(self):
    assert normalizar_doi(f"https://doi.org/{DOI_TESIS}") == DOI_TESIS
```

Entrada.

```
normalizar_doi('https://doi.org/10.3390/educsci11040173')
```

Salida.

```
'10.3390/educsci11040173'
```

Resultado. PASSED

test_quita_el_prefijo_dx_doi_org

Descripción. Se elimina el prefijo http://dx.doi.org/ del DOI.

Código.

```
def test_quita_el_prefijo_dx_doi_org(self):
    assert normalizar_doi(f"http://dx.doi.org/{DOI_TESIS}") == DOI_TESIS
```

Entrada.

```
normalizar_doi('http://dx.doi.org/10.3390/educsci11040173')
```

Salida.

```
'10.3390/educsci11040173'
```


Resultado. PASSED

test_quita_el_prefijo_doi_dos_puntos

Descripción. Se elimina el prefijo doi: del DOI.

Código.

```
def test_quita_el_prefijo_doi_dos_puntos(self):  
    assert normalizar_doi(f"doi:{DOI_TESIS}") == DOI_TESIS
```

Entrada.

```
normalizar_doi('doi:10.3390/educsci11040173')
```

Salida.

```
'10.3390/educsci11040173'
```

Resultado. PASSED

test_pasa_a_minusculas

Descripción. La normalización pasa el DOI a minúsculas, ya que el DOI es insensible a mayúsculas.

Código.

```
def test_pasa_a_minusculas(self):  
    assert normalizar_doi("10.48550/arXiv.2602.05930") ==  
    "10.48550/arxiv.2602.05930"
```

Entrada.

```
normalizar_doi('10.48550/arXiv.2602.05930')
```

Salida.

```
'10.48550/arxiv.2602.05930'
```

Resultado. PASSED

test_recorta_espacios_y_barra_final

Descripción. La normalización recorta espacios circundantes y la barra final.

Código.

```
def test_recorta_espacios_y_barra_final(self):  
    assert normalizar_doi(f" {DOI_TESIS}/ ") == DOI_TESIS
```

Entrada.

```
normalizar_doi(' 10.3390/educsci11040173/ ')
```

Salida.

```
'10.3390/educsci11040173'
```

Resultado. PASSED

test_sin_doi_devuelve_cadena_vacia

Descripción. Un DOI nulo o vacío devuelve cadena vacía, no None.

Código.

```
def test_sin_doi_devuelve_cadena_vacia(self):
    assert normalizar_doi(None) == ""
    assert normalizar_doi("") == ""
```

Entrada.

```
1) normalizar_doi(None)
2) normalizar_doi('')
```

Salida.

```
1) ''
2) ''
```

Resultado. PASSED

test_dos_escrituras_del_mismo_doi_normalizan_igual

Descripción. El mismo DOI escrito como URL en mayúsculas y como identificador simple normaliza al mismo valor, de modo que resuelve a una única fuente.

Código.

```
def test_dos_escrituras_del_mismo_doi_normalizan_igual(self):
    # El mismo paper citado con URL y sin ella debe resolver a la misma fuente.
    assert normalizar_doi(f"https://doi.org/{DOI_TESIS.upper()}") ==
    normalizar_doi(DOI_TESIS)
```

Entrada.

```
1) normalizar_doi('https://doi.org/10.3390/EDUCSCI11040173')
2) normalizar_doi('10.3390/educsci11040173')
```

Salida.

```
1) '10.3390/educsci11040173'
2) '10.3390/educsci11040173'
```

Resultado. PASSED

7.7.2. TestDOIParaChunks

test_construye_la_clave_del_vector_store

Descripción. La clave del almacén vectorial se construye sustituyendo puntos y barras del DOI por guiones bajos.

Código.

```
def test_construye_la_clave_del_vector_store(self):
    assert doi_para_chunks(DOI_TESIS) == "10_3390_educsci11040173"
```

Entrada.

```
doi_para_chunks('10.3390/educsci11040173')
```

Salida.

```
'10_3390_educsci11040173'
```

Resultado. PASSED

test_la_clave_no_contiene_barras_ni_puntos

Descripción. La clave generada no contiene barras ni puntos, caracteres no admitidos como identificador.

Código.

```
def test_la_clave_no_contiene_barras_ni_puntos(self):
    clave = doi_para_chunks("10.48550/arXiv.2602.05930")
    assert "/" not in clave and "." not in clave
```

Entrada.

```
doi_para_chunks('10.48550/arXiv.2602.05930')
```

Salida.

```
'10_48550_arXiv_2602_05930'
```

Resultado. PASSED

test_dois_distintos_producen_claves_distintas

Descripción. Dos DOI distintos producen claves distintas, sin colisión entre artículos.

Código.

```
def test_dois_distintos_producen_claves_distintas(self):
    assert doi_para_chunks(DOI_TESIS) != doi_para_chunks("10.18060/26528")
```

Entrada.

```
1) doi_para_chunks('10.3390/educsci11040173')
2) doi_para_chunks('10.18060/26528')
```

Salida.

```
1) '10_3390_educsci11040173'
2) '10_18060_26528'
```

Resultado. PASSED

test_la_clave_es_estable_entre_llamadas

Descripción. La clave de un mismo DOI es estable entre llamadas sucesivas.

Código.

```
def test_la_clave_es_estable_entre_llamadas(self):
    assert doi_para_chunks(DOI_TESIS) == doi_para_chunks(DOI_TESIS)
```

Entrada.

```
1) doi_para_chunks('10.3390/educsci11040173')
2) doi_para_chunks('10.3390/educsci11040173')
```

Salida.

```
1) '10_3390_educsci11040173'
2) '10_3390_educsci11040173'
```

Resultado. PASSED

7.8. limpiar_fragmento

Archivo de pruebas	Módulo bajo prueba	Pruebas
tests/unit/test_texto.py	app/core/texto.py	8

limpiar_fragmento — saneo del markdown extraído del PDF.

Es la función que garantiza que el índice vectorial, el juez LLM, las cartillas y el Excel vean exactamente el mismo texto corrido. Se prueba con fragmentos tomados del markdown real de la tesis.

7.8.1. TestLimpiarFragmento

test_quita_las_negritas_del_markdown

Descripción. El saneo elimina los asteriscos de negrita del markdown.

Código.

```
def test_quita_las_negritas_del_markdown(self):
    assert limpiar_fragmento("***Jurado evaluador:**") == "Jurado evaluador:"
```

Entrada.

```
limpiar_fragmento('***Jurado evaluador:**')
```

Salida.

```
'Jurado evaluador:'
```

Resultado. PASSED

test_quita_los_encabezados

Descripción. El saneo elimina las almohadillas de encabezado del markdown.

Código.

```
def test_quita_los_encabezados(self):
    assert limpiar_fragmento("## 7. Referencias bibliográficas") == "7. Referencias bibliográficas"
```

Entrada.

```
limpiar_fragmento('## 7. Referencias bibliográficas')
```

Salida.

```
'7. Referencias bibliográficas'
```

Resultado. PASSED

test_colapsa_los_salto_de_linea_en_texto_corrido

Descripción. Los saltos de línea se colapsan en un único espacio, dejando el fragmento como texto corrido.

Código.

```
def test_colapsa_los_saltos_de_linea_en_texto_corrido(self):
    assert limpiar_fragmento("Presidente:\n\nSecretario:\n\nVocal:") ==
    "Presidente: Secretario: Vocal:"
```

Entrada.

```
limpiar_fragmento('Presidente:\n\nSecretario:\n\nVocal:')
```

Salida.

```
'Presidente: Secretario: Vocal:'
```

Resultado. PASSED

test_elimina_los_separadores_horizontales

Descripción. Los separadores horizontales del markdown se eliminan del fragmento.

Código.

```
def test_elimina_los_separadores_horizontales(self):
    assert "-----" not in limpiar_fragmento("Texto\n\n-----\n\nMás texto")
```

Entrada.

```
limpiar_fragmento('Texto\n\n-----\n\nMás texto')
```

Salida.

```
'Texto Más texto'
```

Resultado. PASSED

test_elimina_los_caracteres_invisibles_del_pdf

Descripción. Los caracteres invisibles heredados del PDF, como el espacio de ancho cero y el guion suave, se eliminan.

Código.

```
def test_elimina_los_caracteres_invisibles_del_pdf(self):
    assert limpiar_fragmento("GraphRAG- Auditor") == "GraphRAG Auditor"
```

Entrada.

```
limpiar_fragmento('Graph\u200bRAG\xad Auditor')
```

Salida.

```
'GraphRAG Auditor'
```

Resultado. PASSED

test_una_cadena_vacia_no_rompe

Descripción. Una cadena vacía o nula devuelve cadena vacía, sin lanzar excepción.

Código.

```
def test_una_cadena_vacia_no_rompe(self):
    assert limpiar_fragmento("") == ""
    assert limpiar_fragmento(None) == ""
```

Entrada.

```
1) limpiar_fragmento('')
2) limpiar_fragmento(None)
```

Salida.

```
1) ''
2) ''
```

Resultado. PASSED

test_el_markdown_real_de_la_tesis_queda_en_una_sola_linea

Descripción. Un fragmento del markdown real del documento de prueba queda en una sola línea, sin marcas de negrita ni espacios dobles.

Código.

```
def test_el_markdown_real_de_la_tesis_queda_en_una_sola_linea(self, texto_tesis):
    crudo = texto_tesis[texto_tesis.find("***"):][:400]
    limpio = limpiar_fragmento(crudo)
    assert "\n" not in limpio
    assert "***" not in limpio
    assert " " not in limpio
```

Entrada.

```
limpiar_fragmento(<cadena de 400 caracteres> '**Jurado
    evaluador:**\n\nPresidente:\n\nSecretario:\n\nVocal:\n\n\nApellidos
Nombres\n\nApellidos
    Nombres\n\nAsesor:\n\nCueva Chavez  Walter\n\n\n[Código Orcid:
    https://orcid.org/0000-0001-9660-6354](https://orcid.org/0000-0001-9660-
6354)\n\nTRUJILLO -
    PERÚ\n\n2026\n\nFecha de sust'...)
```

Salida.

```
<cadena de 368 caracteres> 'Jurado evaluador: Presidente: Secretario: Vocal: Apellidos
Nombres
    Apellidos  Nombres Asesor: Cueva Chavez  Walter [Código Orcid:
    https://orcid.org/0000-0001-9660-6354](https://orcid.org/0000-0001-9660-6354) TRUJILLO
-
    PERÚ 2026 Fecha de sustentación: MARCAR '...
```

Resultado. PASSED

test_no_inventa_ni_pierde_palabras

Descripción. El saneo no añade ni elimina palabras: solo limpia el formato.

Código.

```
def test_no_inventa_ni_pierde_palabras(self):
    crudo = "***GraphRAG** mejora\n\nla recuperación"
    assert limpiar_fragmento(crudo) == "GraphRAG mejora la recuperación"
```

Entrada.

```
limpiar_fragmento('***GraphRAG** mejora\n\nla recuperación')
```

Salida.

```
'GraphRAG mejora la recuperación'
```

Resultado. PASSED